

OpenCV Ch5

Image Processing and Convolution

Topic I. Similarity Measure (相似度量測)

3.0 Image Processing for Deep Learning: Filter, Correlation and Convolution 3.1~3.3

National Cheng Kung University
Dept. of Computer Science and Information Engineering
Robotics Lab.

國立成功大學 資訊工程系 機器人實驗室

(Jenn-Jier James Lien / 連震杰)
<http://robotics.csie.ncku.edu.tw>
jjlien@csie.ncku.edu.tw
0932 962671



國立成功大學



Department of Computer Science and Information Engineering
Robotics Lab

Content: Topic1. Similarity Measure (相似度量測)

1.0 Computer Vision - Camera Model: Between 3D and 2D

1.1 2D Image

1.2 Camera Model Btw 3D Object and 2D Image

1.3 From Linear Combination to Homography $Ax=b$ (Vector, Matrix)

2.0 Computer Vision (CV) and AI: Similarity Measure = Loss Function

2.1 Sum-of-Squared Differences (SSD) and Mean Squared Error (MSE)

2.2 Normalized Correlation Coefficient (NCC) and Cross Entropy (CE)

2.3 AI Bayesian Decision Rule - Maximum a Posteriori (MAP) Probability with Gaussian Model

3.0 Image Processing for Deep Learning: Filter, Correlation and Convolution

3.1 Low-Pass Filter for Denoise and High-Pass Filter for Edge Detection (Feature Extraction) via Convolution

3.2 Correlation Vs. Convolution

3.3 Multi-Resolution - Downsampling (Encoder) and Upsampling (Decoder)

3.0 Background

- 1) Low-Pass Filter → **Gaussian Model**
- 2) Linear Combination → Eigenspace → GAN
- 3) Bayes Theory → Loss Function (Similarity Measure) → Optimization



B1-1.1 Filters: Low-Pass Vs. High-Pass

1. Low-pass filter: Smoothing filter to denoise

- 低通濾波一般又稱為平滑濾波(Smoothing filter)，通常用於模糊化與減少高頻雜訊。
。最常見的此類濾波有平均濾波(Averaging filter)與高斯平滑濾波 (Gaussian filter)。
- Original energy, such as sum of each pixel gray value $I^2(x,y)$ over entire image, will change. So it is hard to recover.

$\sin\theta$

$\cos\theta$

θ

	x -1	0	+1
y -1	1/9	1/9	1/9
0	1/9	1/9	1/9
+1	1/9	1/9	1/9

1/16	2/16	1/16
2/16	4/16	2/16
1/16	2/16	1/16

Sum of filter coefficients = 1

$h[u,v]$
3x3 (a) 平均濾波

$h[u,v]$
3x3 (b) 高斯平滑濾波

2. High-pass filter: Extract high frequency texture

- 高通濾波器是用來突顯影像中高頻特徵 feature 的成分，在影像較為模糊的情況可讓影像較為清晰。

-1/9	-1/9	-1/9
-1/9	8/9	-1/9
-1/9	-1/9	-1/9

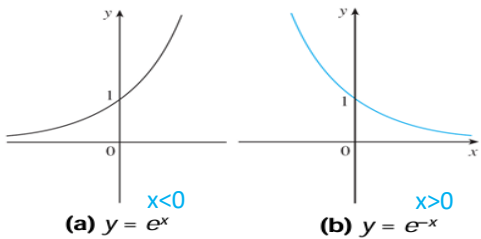
$h[u,v]$
3x3 高通濾波係數

Sum of filter coefficients = 0

→ threshold

B1-1.1 Low-Pass Smoothing Filter: Gaussian Filter

Gaussian Model 1) Parameter $N(u, \sigma^2)$, 2) Non-Parameter
→ Markov Model → Deep Learning (Network Model)



Gaussian Filter

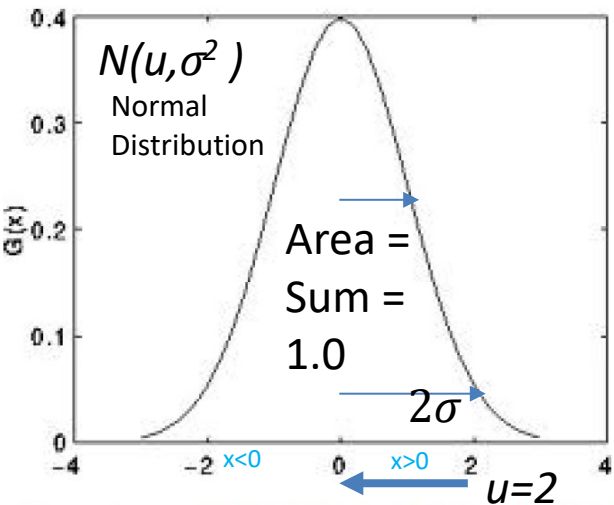
- Probably the most useful filter (although not the fastest). Gaussian filtering is done by convolving each point in the input array with a Gaussian kernel and then summing them all to produce the output array.
- Just to make the picture clearer, remember how a 1D Gaussian kernel look like?

e: Activation function

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Normalization term: to between 0.0 ~ 1.0

Exponential
→ like Sigmoid activation function
→ Push y value up closer to 1.0 or
Press y value down closer to 0.0
→ Won't cause confusion decision =
threshold y=0.5



近似3*3 Gaussian Filter

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

generalized weighted smoothing filter

Assuming that an image is 1D, you can notice that the pixel located in the middle would have the biggest weight.
The weight of its neighbors decreases as the spatial distance between them and the center pixel increases.

Remember that a 2D Gaussian can be represented as :

$$S(t) = \frac{1}{1 + e^{-t}}$$

$$G_0(x, y) = A e^{-\frac{(x - \mu_x)^2}{2\sigma_x^2} - \frac{(y - \mu_y)^2}{2\sigma_y^2}}$$

$$A = \frac{1}{2\pi\sigma^2}$$

where μ is the mean (the peak) and σ represents the ~~variance~~ Standard deviation (per each of the variables x and y)

B1-1.1 Gaussian Model $N(\mu, \sigma^2)$ (1/2)

1. Gaussian Distribution (Normal Distribution):

1) 當樣本為1-D (univariate)，高斯分佈遵守機率密度函數 (Probability Density Function - PDF):

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

$$x \in \{x_1, x_2, \dots, x_n\}$$

$$\mu = \frac{1}{n} \sum_{i=1}^n x_n: \text{mean (平均值或期望值): Average}$$

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}: \text{standard deviation (標準差)}$$

• 右圖為不同平均數與標準差，所對應的機率密度函數曲線。

$N(u, \sigma^2)$
Normal Distribution

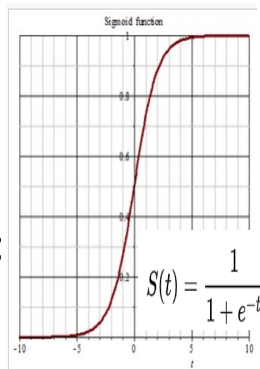
• Gaussian Distribution 物理意義:

- distance of x from μ
- degree of spread

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Weight(kg)
10kg
10cm
Height(cm)

- 位移: 回歸基準
- Exponential: 使大小值拉開
- 除標準差: 使 term 統一單位

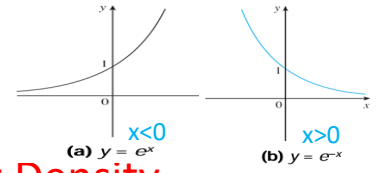


Like Activation Function of deep learning

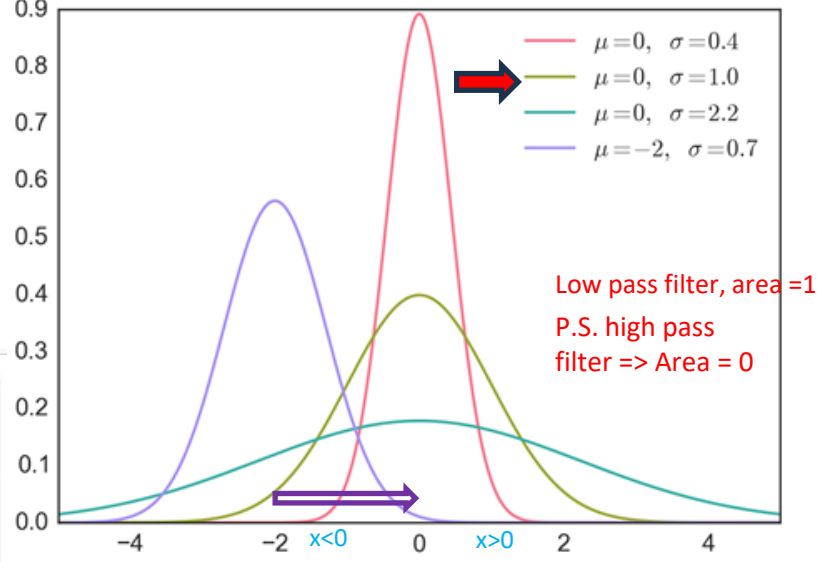
Affine translation:
- "Shift/translation,
- σ normalization/scaling

Normalization term: to between 0.0 ~ 1.0

- SSE $\sum(y - y')^2$
 y : Ground Truth
 y' : Estimation result

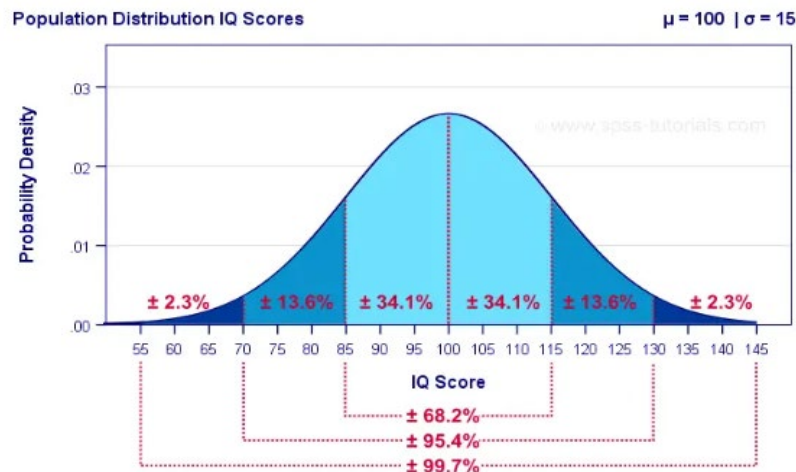


不同平均數與標準差，所對應的機率密度函數曲線



- μ : Depend on Center-Position by μ (average),
- σ Depend on height and degree of spread by σ (standard deviation)

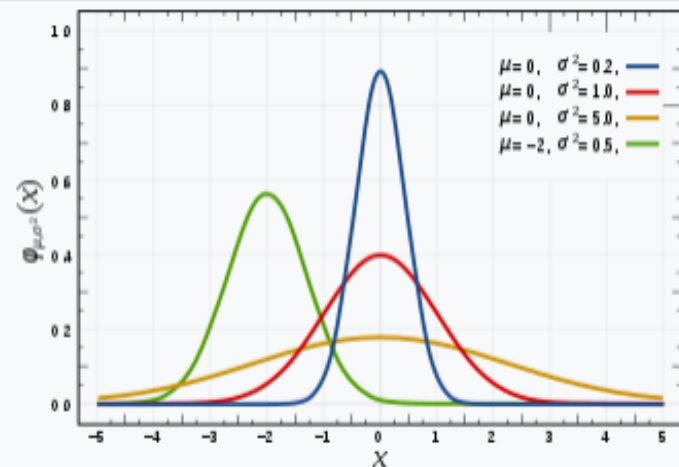
- a) Gaussian distribution (Probability)
 $m \pm \sigma$ (68%), $m \pm 2\sigma$ (95%?), $m \pm 3\sigma$ (99%)
- b) Gaussian Mixture Model (GMM)
➔ Weighted GMM estimated by EM



Notation	$\mathcal{N}(\mu, \sigma^2)$
Parameters	$\mu \in \mathbb{R}$ = mean (location) $\sigma^2 \in \mathbb{R}_{>0}$ = variance (squared scale)
Support	$x \in \mathbb{R}$
PDF	$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$
CDF	$\Phi\left(\frac{x-\mu}{\sigma}\right) = \frac{1}{2} \left[1 + \operatorname{erf}\left(\frac{x-\mu}{\sigma\sqrt{2}}\right) \right]$

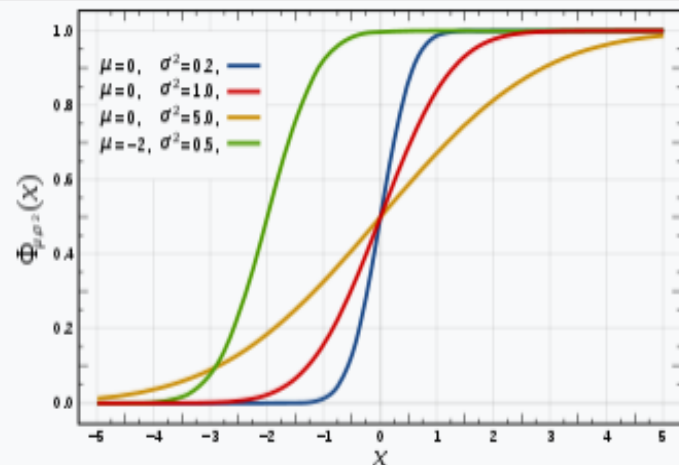
Normal distribution

Probability density function



The red curve is the *standard normal distribution*

Cumulative distribution function



B1-1.1 Gaussian Model $N(\mu, \sigma^2)$ (2/2)

2) 當樣本為 n-D (multivariate), x 為 vector (向量化), 高斯分佈:

$$P(x) = \frac{1}{\sqrt{(2\pi)^D |\Sigma|}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

μ : 平均值 ($d \times 1$)

$$\Sigma = \begin{bmatrix} (x_{11} - \mu)^2 & \cdots & (x_{1D} - \mu)^2 \\ \vdots & \ddots & \vdots \\ (x_{D1} - \mu)^2 & \cdots & (x_{DD} - \mu)^2 \end{bmatrix}$$

D: 維度

$|\Sigma|$: 協方差的行列式

協方差矩陣(共變異數矩陣)

- covariance matrix

($d \times d$ symmetric matrix)

Note: 如果沒有 $-\mu$ 就是 correlation

tommy

- Exponential 的物理意義:

$$\exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

Mahalanobias Distance

- 表示數據的協方差距離

1) μ : Translation

2) Σ : Scaling & Rotation

→ invariant

- If the variables are independent:

$$P(x) = \prod_i \frac{1}{\sigma_i \sqrt{2\pi}} \exp\left[-\frac{(x_i - \mu_i)^2}{2\sigma_i^2}\right]$$

- SSD $\Sigma(y-y')^2$

y : Ground Truth

y' : Estimation result

Cannot compare at

difference unit or space

Can compare at

difference unit or space

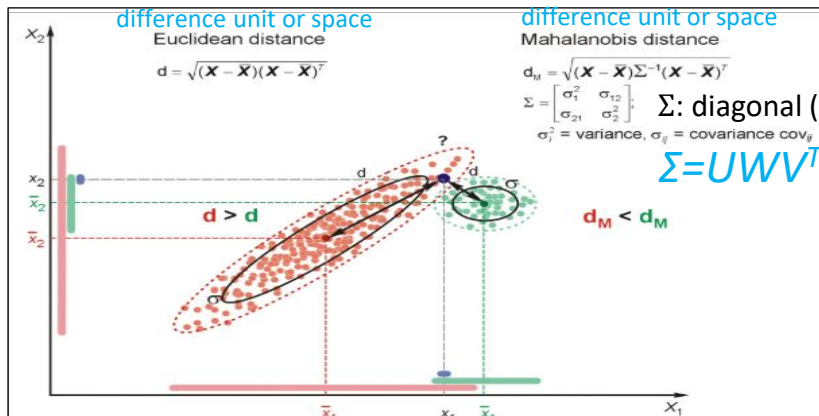
$$d_M = \sqrt{(X - \bar{X})^T \Sigma^{-1} (X - \bar{X})}$$

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{21} & \sigma_2^2 \end{bmatrix}$$

Σ : diagonal (只有對角線上有值)

σ_i^2 : variance, σ_{ij} : covariance cov_{ij}

$$\Sigma = U W U^T$$



'Euclidan Distance' VS 'Mahalanobis Distance'

tommy

Mahalanobias Distance

- Distance of Data Covariance
- The stronger the correlation between variables, the greater the value.

• Using Gaussian function At the Multivariate and Vector
Similarity measure → 1/Distance Measure

1) SSD / MSD problem => Without Normalization Sigma

2) Mahalanobis Distance => With Normalization Sigma + square root

$$\Sigma = U W U^T$$

• PCA (Principal Component Analysis)

- Gaussian model

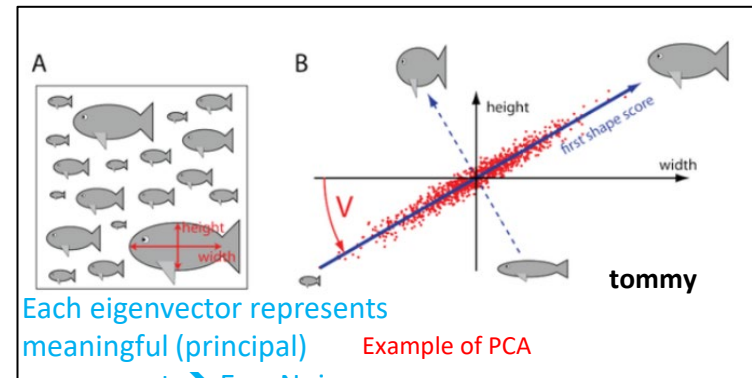
- because it is from Covariance matrix $N(u, \sigma^2)$

1) Covariance matrix ($-u$): Translation

2) Eigenvector U (& V): Direction = Rotation,

3) Eigenvalue W : Variance = Scaling

(present different covariance by PCA)



Each eigenvector represents
meaningful (principal)

Example of PCA

component → Eye, Noise,
month...

Normalization to (0.0~1.0)

1) Gaussian

2) Softmax at DL

2.1) Range of element => (0, 1)

2.2) Sum of elements => 1

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K.$$

PCA

SVD: $(A-m) = UWV^T$; $(A-m)V=UW$; **Gaussian Model** $N(m,\sigma^2)$

where $\frac{u}{|u|}$ $|u|$
U: Orthonormal Matrix, Eigenvector, rotation
W: Scale Matrix, Eigenvalue, Standard Deviation
V: Rotation (Orthonormal) Matrix

$$u = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_{N^2} \end{bmatrix} = \begin{bmatrix} e_{11} & e_{12} & \cdots & e_{1N^2} \\ e_{21} & e_{22} & \cdots & e_{2N^2} \\ \vdots & \vdots & \cdots & \vdots \\ e_{N^21} & e_{N^22} & \cdots & e_{N^2N^2} \end{bmatrix}$$

$$W = \begin{bmatrix} \lambda_1^2 & & & 0 \\ & \lambda_2^2 & & \\ & & \ddots & \\ 0 & & & \lambda_{N^2}^2 \end{bmatrix}$$

$$u_l^T u_k = e_l^T e_k = \delta_{lk} = \begin{cases} 1, & \text{if } l=k \\ 0, & \text{otherwise} \end{cases}$$

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_M \geq \lambda_{M+1} \geq \dots \geq \lambda_{N^2}$$

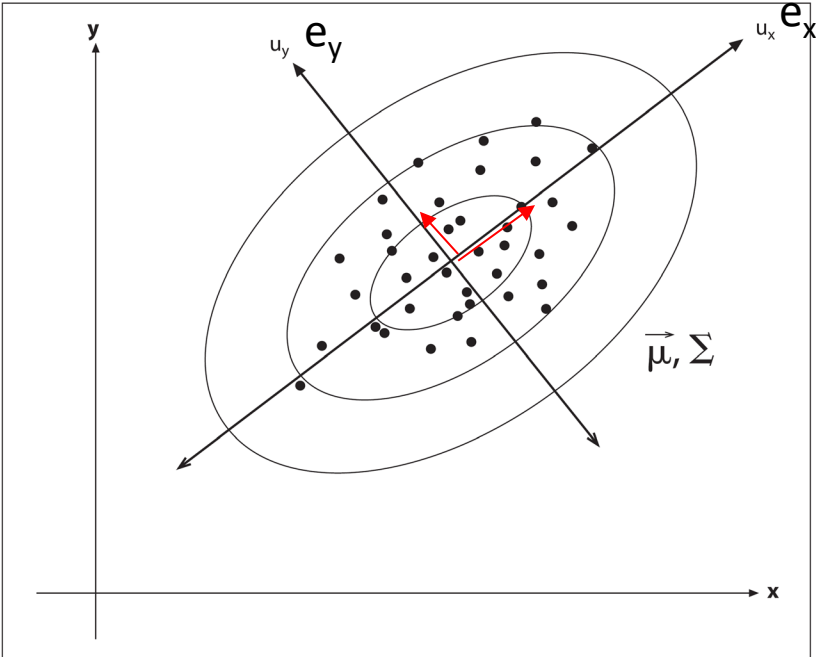


Figure 3-5. A distribution of points in two dimensions with superimposed ellipsoids representing Mahalanobis distances of 1.0, 2.0, and 3.0 from the distribution's mean

B1-1.2 Gaussian Model $N(\mu, \sigma^2)$ - Gaussian Mixture Model (1/3)

2. Gaussian Mixture Model (GMM, $\cos\theta$): $\rightarrow$ Wavelets ($\sin\theta + \cos\theta$)

1) It can be considered as a combination of K -Single Gaussian Mixture

2) For any number of patterns, with Single Gaussian Compared with Vector Quantization, GMM has excellent approximation.

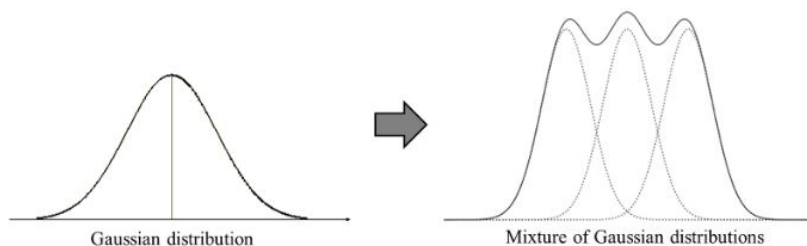
(1) Single Gaussian Mixture: Using an average direction Quantity represents the central location of a group of samples in vector space and approximates the distribution of these samples in vector space using a covariable matrix.

(2) Vector Quantization : represented by several important positions. The whole vector space, but the model itself does not describe the size and shape of the sample in space.

3) GMM uses multiple SINGLE Gaussian Mixture to represent the distribution of feature vectors.

- It not only accurately records the type of sample and its location in the vector space, but also describes the size and shape of these categories in space,

- so GMM is suitable to describe the distribution of feature vectors in the color space.



B1-1.2 Gaussian Model $N(\mu, \sigma^2)$ - Gaussian Mixture Model (2/3)

4) GMM 模型:

$$P(x|\theta) = \sum_i^K \alpha_i G(x|\theta_i)$$

$\theta = (\widetilde{\mu}_k, \widetilde{\sigma}_k, \widetilde{\alpha}_k)$: 每個子模型的期望, 協方差, 在GMM中的機率
 K : Gaussian Model 的數量, $i = 1, 2, \dots, K$
 a_k : 第 k 個子模型的權重, $a_k > 0, \sum_{i=1}^K a_k = 1$
 $G(x|\theta_k)$: 是第 k 個子模型的高斯密度函數, $\theta_k = (\mu_k, \sigma_k^2)$

- GMM (low pass filter, Sum **area=1**) Vs. Wavelets (high (Sum **area=0**)+ low pass filters)

→ Fourier Series

- a. Any distribution can be the **linear combination** of GMM
→ Wavelets is better
- b. Linear combination → $Ax = b$ Homogenous matrix
→ Optimization process to solve parameters

→ Signal process: Fourier series, wavelets

5) Signal Process: Fourier Series → Linear Combination

圖示: $s(t) = a(t) + b(t) + c(t) + d(t)$

Fourier series, amplitude-phase form

$$s_N(x) = \frac{A_0}{2} + \sum_{n=1}^N A_n \cdot \cos\left(\frac{2\pi}{P}nx - \varphi_n\right) \quad (\text{Eq.1})$$

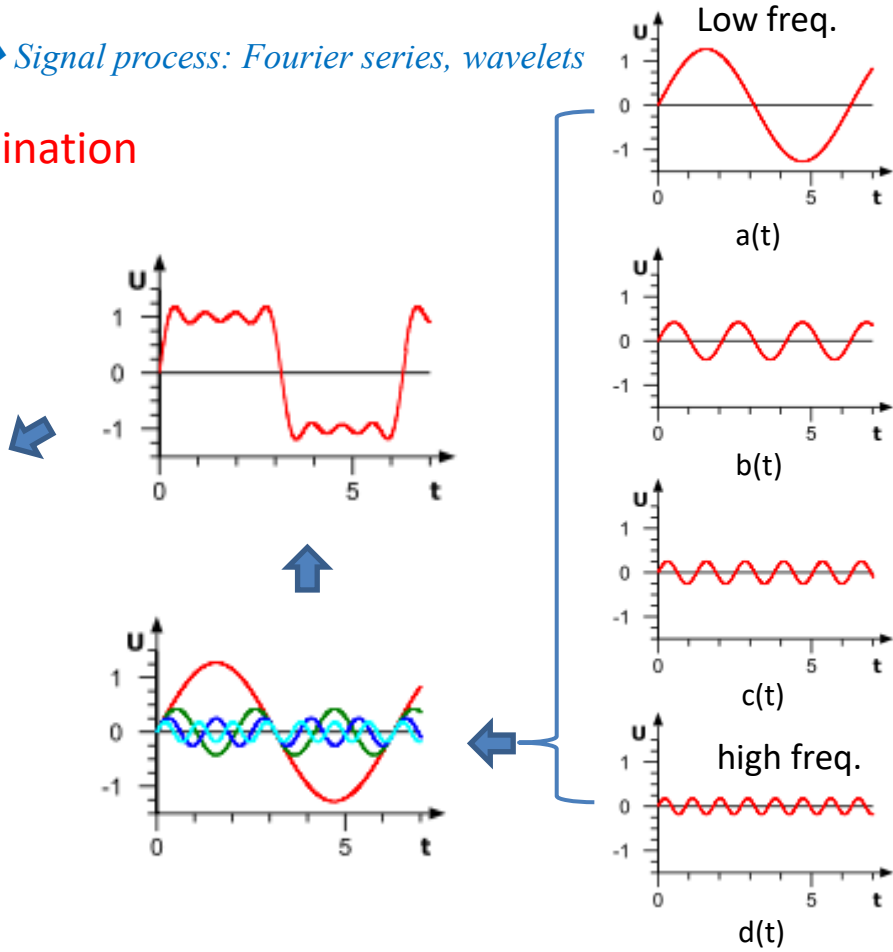
Fourier series, sine-cosine form

$$s_N(x) = \frac{a_0}{2} + \sum_{n=1}^N \left(a_n \cos\left(\frac{2\pi}{P}nx\right) + b_n \sin\left(\frac{2\pi}{P}nx\right) \right) \quad (\text{Eq.4})$$

Fourier series, exponential form

$$s_N(x) = \sum_{n=-N}^N c_n \cdot e^{i2\pi nx/P} \quad (\text{Eq.5})$$

$$\begin{cases} x' = ax + by + t_x \\ y' = cx + dy + t_y \end{cases}$$



B1-1.2 Gaussian Model $N(\mu, \sigma^2)$ - Gaussian Mixture Model (3/3)

4) 模型(Model):

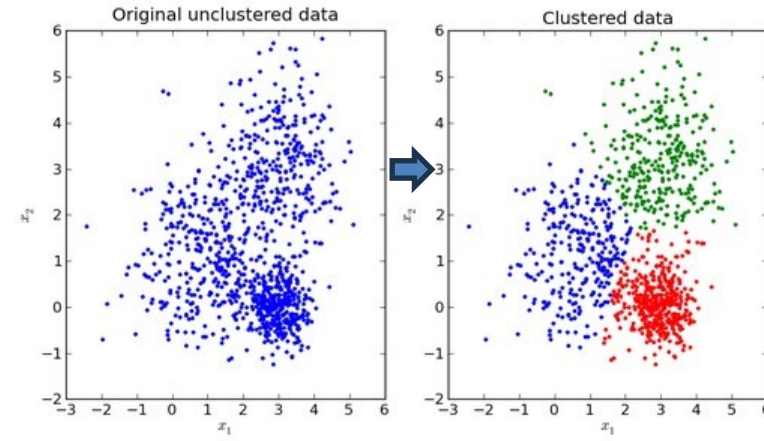
$$P(x|\theta) = \sum_i^K \alpha_i G(x|\theta_i)$$

$\theta = (\widetilde{\mu}_k, \widetilde{\sigma}_k, \widetilde{\alpha}_k)$: 每個子模型的期望, 協方差, 在GMM中的機率
(Expectation, covariance, probability in GMM for each sub-model)

K : Gaussian Model 的數量, $i = 1, 2, \dots, K$ (numbers of gaussian models)

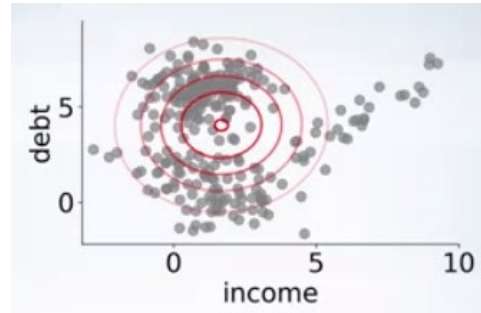
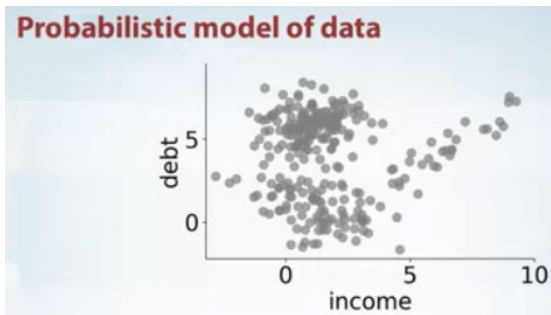
α_k : 第 k 個子模型的權重(概率), $\alpha_k > 0$, $\sum_{i=1}^K \alpha_k = 1$ (model weight of K)

$G(x|\theta_k)$: 是第 k 個子模型的高斯密度函數, $\theta_k = (\mu_k, \sigma_k^2)$
(Gaussian density function of K -th)

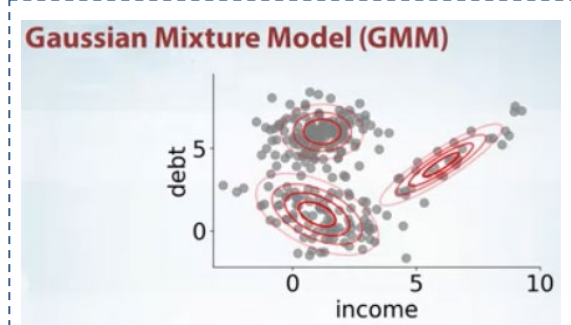


登隔熱分佈及走勢 by
MeanShift or CamShift

5) 圖示:



[Single Gaussian Mixture]



[GMM]

Q : How to know K (Number of Gaussian Model)?

A : Vector Quantization: Clustering algorithm

GMM (low pass filter, **area=1**)

Wavelets (high(**area=0**)+ low pass filters (**area=1**))

B1-2 Linear Combination: Basis Function

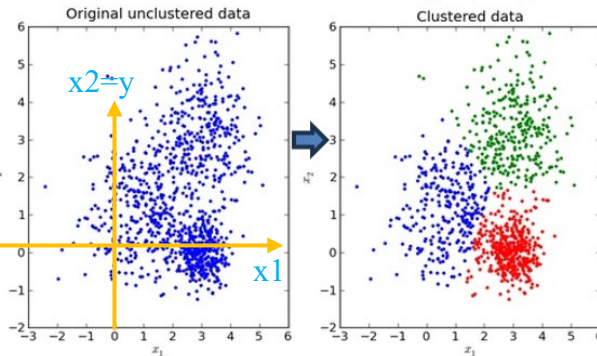
Why Eigenvector and Eigenvalues are important?

→ Basis function of linear combination

● Homogenous Matrix A , $b=Ax$

Linear Combination $y=w1*x1+w2*x2+b=$

$$f(x) = a_0 + \sum_{n=1}^{\infty} \left(a_n \cos \frac{n\pi x}{L} + b_n \sin \frac{n\pi x}{L} \right)$$



- Basic function: cos or sin, (eigenvector, wavelets)

→ orthonormal vector = orthogonal + normalization (unit vector)

- Parameter: a , b , ...

Point or sample in high dimension:

20x20 pixels face image is a

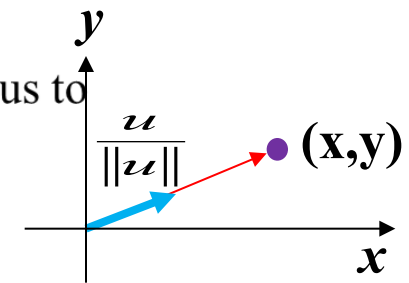
point/sample/vector in 400 dimensions

□ A simplistic view

➤ Homogenous coordinates (matrix) are a mechanism that allows us to associate points and vectors in space with vectors in $\mathbb{R}$ Real

Data collection = Collect Samples → Sample Points (distribution)

□ Location/Point $u = \text{Unit vector} * \text{Amplitude} = \frac{u}{||u||} * ||u||$
 = direction = scaling
 = rotation



□ SVD: $(A-m) = UWV^T$; $(A-m)V = UW \frac{u}{||u||}$, Gaussian Model $N(m, \sigma^2)$

where

U : Orthonormal Matrix, Eigenvector, rotation

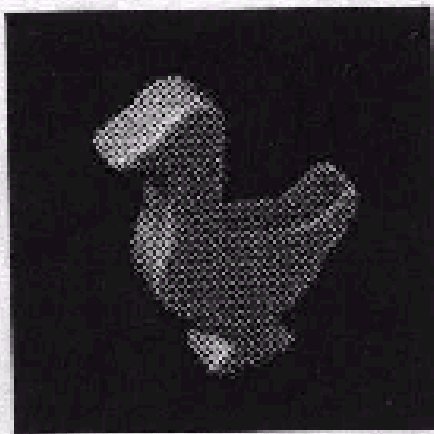
W : Scale Matrix, Eigenvalue, Standard Deviation

V : Rotation (Orthonormal) Matrix

B1-2 Domain Knowledge (manifold):

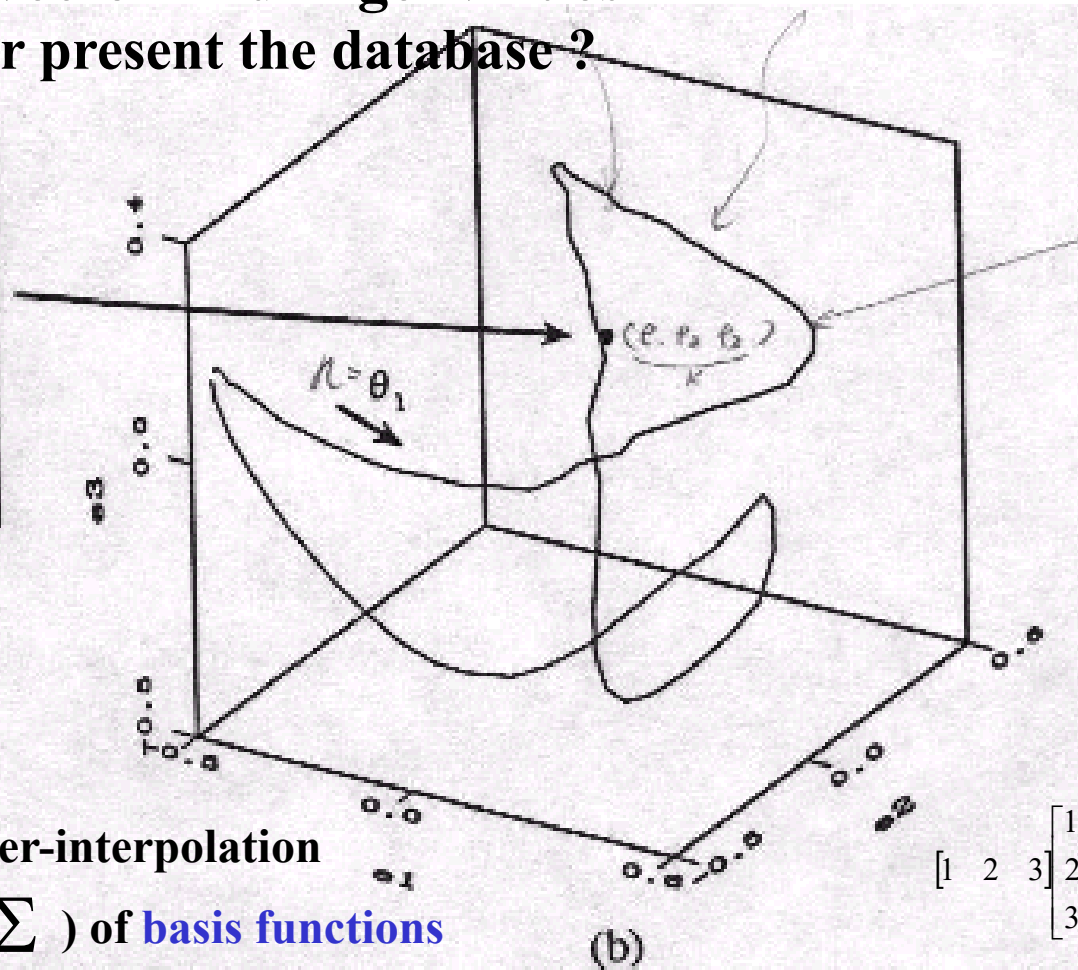
Eigenspace - Eigenvector and Eigenvalues

□ How do you quantize or present the database?



(a)

$$z = [e_1, e_2, e_3]^T (y - c)$$



(b)

Manifold (subspace): Inter-interpolation

Linear Combination ($= \sum$) of **basis functions**

($\Rightarrow$ Homogenous Matrix or Matrix $* [1 \ 1 \ \dots \ 1]^T$):

$$1*v_1 + 1*v_2 + \dots + 1*v_M = \lambda_1*e_1 + \dots + \lambda_M'*e_M + \dots + \lambda_M*e_M$$

given any $v_i = w_1*e_1 + w_2*e_2 + \dots + w_M'*e_M + \dots + w_M*e_M$

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 11 & 12 & 13 \\ 21 & & \\ 31 & & 33 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} 11 & 12 & 13 \\ 21 & & \\ 31 & & 33 \end{bmatrix}$$

Independent Vs. Orthogonal

Eigenvectors, orthonormal vector

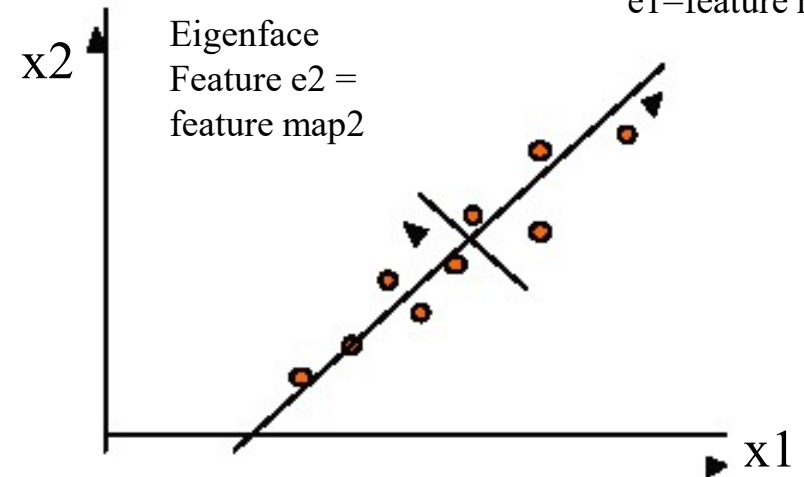
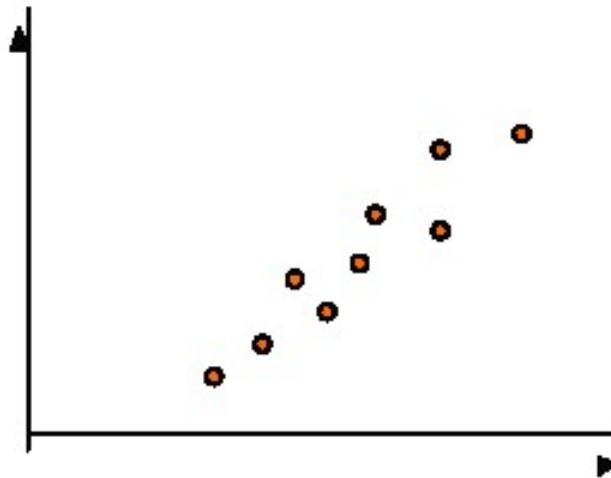
- Find an orthogonal set of basis vectors for the feature space, subject to the requirement that the new features have zero correlation with each other.

eigenvectors

- Dimensional reduction.

□ Affine

Estimate the shortest distance (errors) (point to plane) over all samples



SVD: $(A-m) = UWV^T$; $(A-m)V=UW$; Gaussian Model $N(m, \sigma^2)$

where

U : Orthonormal Matrix, Eigenvector, rotation

W : Scale Matrix, Eigenvalue, Standard Deviation

V : Rotation (Orthonormal) Matrix

$$u = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_{N^2} \end{bmatrix} = \begin{bmatrix} e_{11} & e_{12} & \cdots & e_{1N^2} \\ e_{21} & e_{22} & \cdots & e_{2N^2} \\ \vdots & \vdots & \cdots & \vdots \\ e_{N^2 1} & e_{N^2 2} & \cdots & e_{N^2 N^2} \end{bmatrix}$$

$$W_\lambda = \begin{bmatrix} \lambda_1^2 & & & 0 \\ & \lambda_2^2 & & \\ & & \ddots & \\ 0 & & & \lambda_{N^2}^2 \end{bmatrix}$$

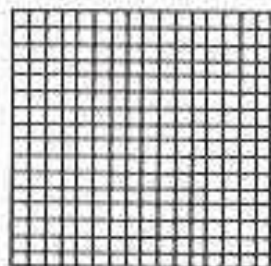
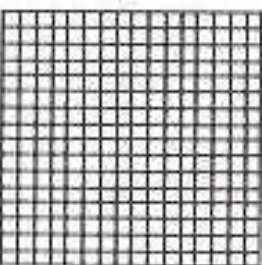
$$u_l^T u_k = e_l^T e_k = \delta_{lk} = \begin{cases} 1, & \text{if } l=k \\ 0, & \text{otherwise} \end{cases}$$

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_M \geq \lambda_{M+1} \geq \dots \geq \lambda_{N^2}$$



Input image

Γ

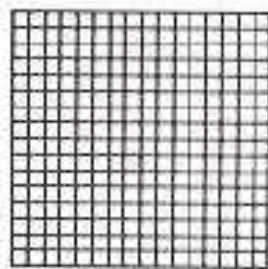


Mean image

Ψ

$$\Phi_i = \Gamma_i - \Psi$$

Φ



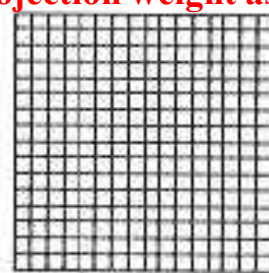
u_i
Encoder
SVD

u_i
Decoder



$$\Omega^T = [\omega_1, \omega_2, \dots, \omega_{M'}]$$

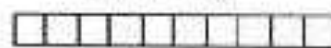
Projection weight as code



$\Phi - \Phi_f$

for face recognition

$$\text{Identity } \varepsilon_k = \|\Omega - \Omega_k\|^2 \quad 1 \leq k \leq M$$

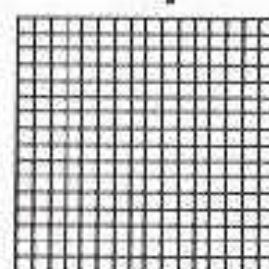


$$\Phi_f = \sum_{i=1}^{M'} \omega_i u_i$$

Projected image

Φ_f

Reconstructed image



$$\varepsilon^2 = \|\Phi - \Phi_f\|^2$$

ε

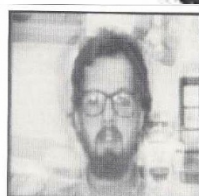
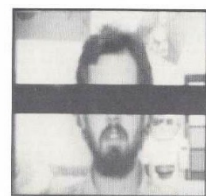
Distance measure

for face detection

Jenn-Jier James Lien

Novel Input Image

Reconstruction



B1-2 Eigenvector Vs. DcGAN

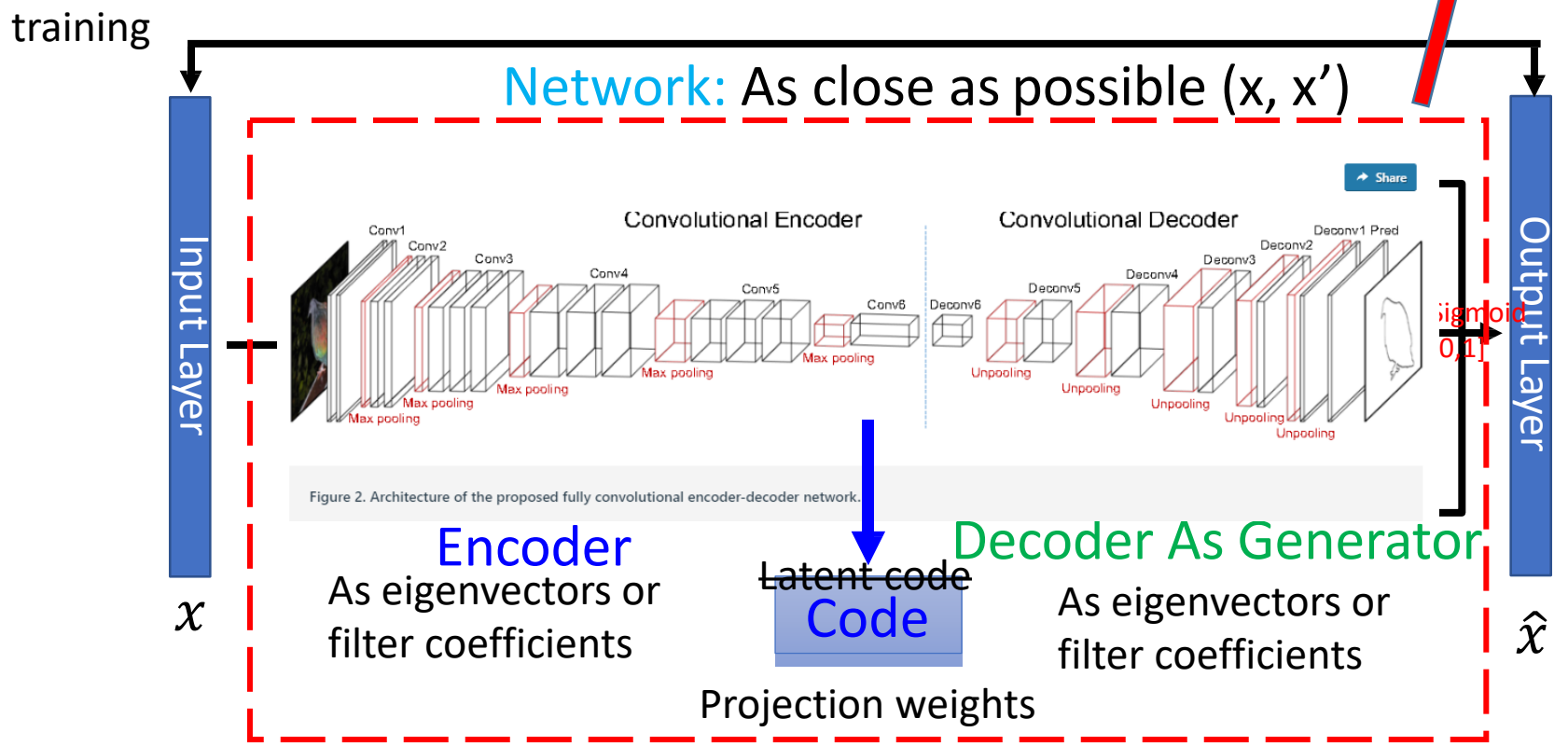
Reconstruction:
1) Eigenspace: Encoder = Decoder
2) GAN: Encoder != Decoder

- 1. Auto encoder: image $x \rightarrow$ code $\rightarrow$ image x' .
- 2. We hope our generator network could input a code then output an image.
=> Similar to the decoder.
- 3. As eigenspace: Eigenvectors and eigenvalues

Analysis
(Vision) Synthesis
(Graphics)

Pre-train objective:
Input real image in Generator, let output and input as close as possible
J: 完整的Generator Network
Pre-train後把Encoder拿掉，成為GAN的Generator

• NN encoder + NN decoder = **a deep network**



B1-2 Eigenvector Vs. DcGAN

Approach and Model Architecture – Training: Generator + Discriminator

■ Eliminating fully connected layers on top of convolutional features.

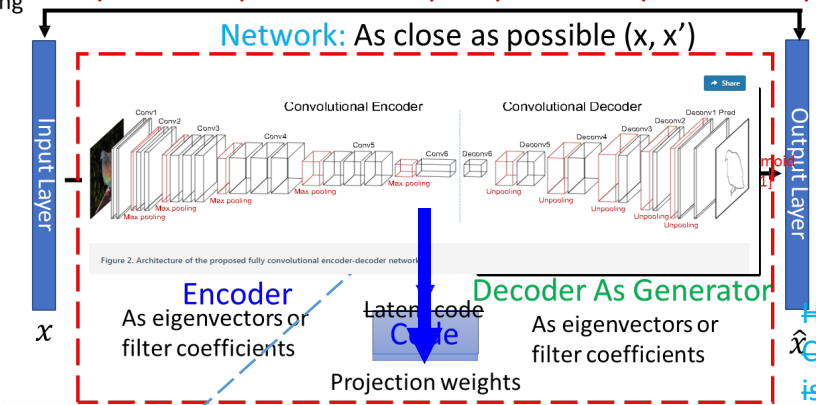
- Global average pooling: increased model stability but hurt convergence speed.
- A middle ground of directly connecting the highest convolutional features to the input and output respectively of the generator and discriminator worked well.
- Generator
 - First layer could be called fully connect: just a matrix multiplication.
 - Reshape the result of first layer into 4 dimensional tensor and used as start of the convolution stack.
- Discriminator: the last convolutional layer is flattened and then fed into single sigmoid[0,1] output.

Reconstruction:

1) Eigenspace: Encoder = Decoder

2) GAN: Encoder != Decoder

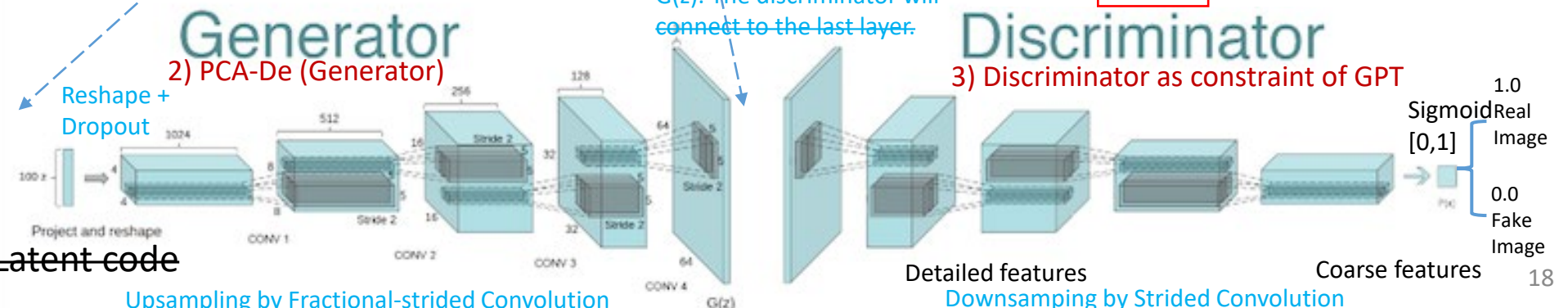
ng As 1) PCA-En (Latent Code) + 2) PCA-De (Generator)



Jenny: in traditional CNN, the feature maps of last convolutional layer are concatenate with fully connect layer.

Here it is the generator:
Conv1~conv4, the last layer is not fully connection lay
 $G(z)$. The discriminator will connect to the last layer.

```
# In: 100
# Out: dim x dim x depth
self.G.add(Dense(dim*dim*depth, input_dim=100))
self.G.add(BatchNormalization(momentum=0.9))
self.G.add(Activation('relu'))
self.G.add(Reshape((dim, dim, depth)))
self.G.add(Dropout(dropout))
```



B1-3 AI Bayesian Decision Rule

Maximum a Posteriori (MAP) Probability with Gaussian Model

1. Maximum a Posteriori (MAP) Probability with Gaussian Model

Decision
1) Posterior Prob. $\propto$ Likelihood Prob. * Priori Prob. Similarity Measure Imbalance, position

2) Likelihood Prob. by Gaussian Model

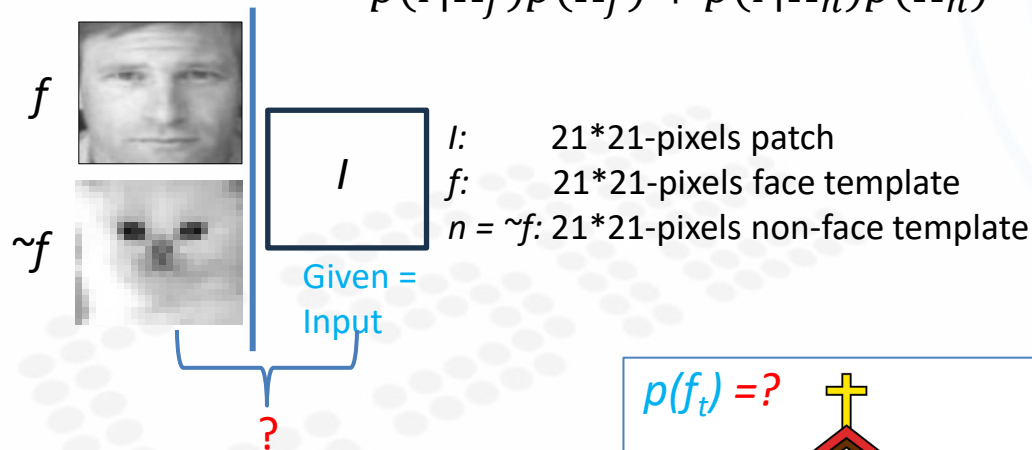
Prob. Of existing a face? MAP: $p(\Omega_f|I) = \frac{p(I|\Omega_f)p(\Omega_f)}{p(I|\Omega_f)p(\Omega_f) + p(I|\Omega_n)p(\Omega_n)}$

> The similarity btw I and f

$$p(I|f) = \frac{1}{(2\pi)^{1/2} |\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(I - f)\Sigma^{-1}(I - f)\right]$$

Covariance matrix $\Sigma = \begin{bmatrix} \sigma_1^2 & 0 & 0 \\ 0 & \sigma_2^2 & 0 \\ 0 & 0 & \sigma_3^2 \end{bmatrix}$

Max: $p(\Omega_n|I) = \frac{p(I|\Omega_n)p(\Omega_n)}{p(I|\Omega_f)p(\Omega_f) + p(I|\Omega_n)p(\Omega_n)}$



4) Loss Function = $p(\Omega_f|I)$

= $p(f_t)$ * Similarity Measure Imbalance

- Similarity Measure:

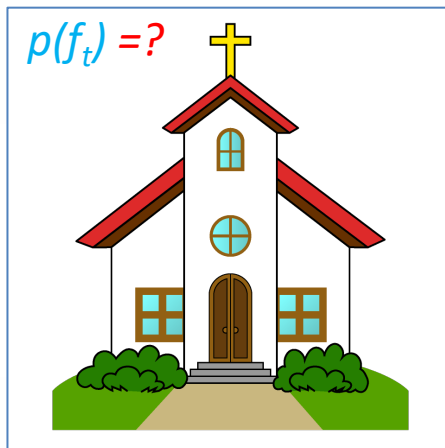
$$\begin{aligned} \min SSD &= \min(I - f)^2 \\ &= \min(I^2 - 2If + f^2) \\ \min SSD &\rightarrow (I^2 + f^2) - 2(If) \end{aligned}$$

$$\therefore \max CC \rightarrow If$$

1,2,...,4,5,6,...8,9
 9,8,...,6,5,4,...2,1

3) Priori Prob. $p(f_t)$ or $p(!f) \neq 1/2$

> Example: position of face appearance probability at time t



Deep Learning:

1) MSE

2) Cross Entropy

min

SSD

9-1 = 8

5-5 = 0

max

correlation

9*1 = 9

5*5 = 25

SSD: Sum of Square Difference

CC: Cross Correlation

B1-3 Homography Estimation $Ax=b$

(1/4)

Distance: norm

$r_{\text{Mahalonobis}}$

$$r_{\text{Mahalonobis}} = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Mahalonobis distance

$$d = \sqrt{(\mathbf{m}_i - \hat{\mathbf{m}}_i)^T (\mathbf{m}_i - \hat{\mathbf{m}}_i)}$$

Euclidean distance = norm 1/2

1. Estimate Homography (H) Between the 3D Model Plane and its 2D Image:

H can be calculated by $\min_H \sum_i \|m_i - \hat{m}_i\|^2$ SSD: Sum of Squared Difference

- Assume 1: m_i is corrupted by Gaussian noise with mean 0 and covariance matrix Λ_{m_i} .

$\Rightarrow$ Probability Density Function (PDF) as likelihood probability:

$$P(\cdot) = \frac{1}{(2\pi)^{N/2} |\Lambda_{m_i}|^{1/2}} \exp \left(-\frac{1}{2} (m_i - \hat{m}_i)^T \Lambda_{m_i}^{-1} (m_i - \hat{m}_i) \right)$$

N : dimension of m_i , $N = 2$ in our case

$\Rightarrow$ Maximum likelihood estimation of H : $\min_H \sum_i (m_i - \hat{m}_i)^T \Lambda_{m_i}^{-1} (m_i - \hat{m}_i)$

- Assume 2: for all i , $\Lambda_{m_i} = \sigma^2 I$.

$$\Rightarrow \min_H \sum_i (m_i - \hat{m}_i)^T \Lambda_{m_i}^{-1} (m_i - \hat{m}_i) = \min_H \sum_i \|m_i - \hat{m}_i\|^2$$

$m' = HM$

- We can obtain H by applying Levenberg-Marquardt algorithm

- An initial guess using close form solution
- Levenberg-Marquardt algorithm

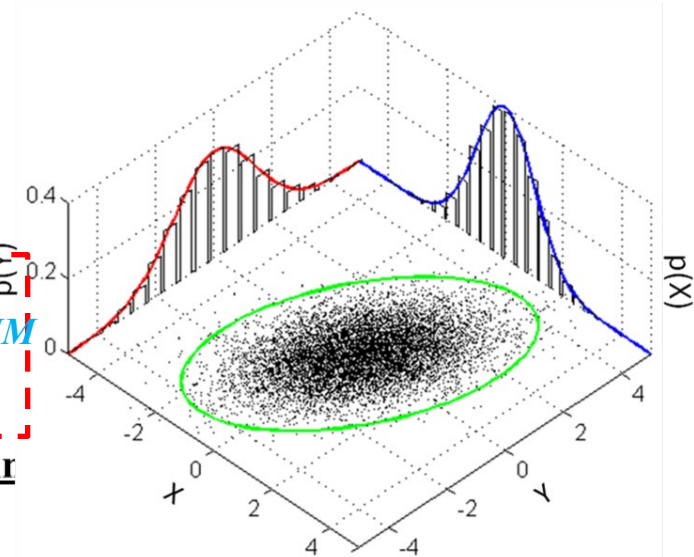
M_i : The 3D model points. (known)

m_i : The 2D image points. (known)

$\hat{m}_i$: Expected image points under the homography H .

$$\hat{m}_i = \begin{bmatrix} u \\ v \end{bmatrix} = \frac{1}{\bar{h}_3^T \hat{M}} \begin{bmatrix} \bar{h}_1^T \hat{M} \\ \bar{h}_2^T \hat{M} \end{bmatrix}$$

?



B1-3 Homogeneous Matrix $Ax=b$: 3D

Camera to 2D Image Coordinate (2/4)

■ Using homogeneous coordinates:

$$\lambda \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \\ \mathbf{1} \end{bmatrix} = \begin{bmatrix} \alpha & \gamma & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R_{11} & R_{21} & R_{31} & T_1 \\ R_{12} & R_{22} & R_{32} & T_2 \\ R_{13} & R_{23} & R_{33} & T_3 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Scale Factor: λ ? Normalize to unit vector [Xc Yc Zc]

Non-linear

$$\begin{cases} x = f \frac{X}{Z} \\ y = f \frac{Y}{Z} \end{cases}$$

Linear

1) Using homogeneous coordinates:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Intrinsic parameter ? Scaling factor ? at camera coordinate

2) Non-linear

$$x = \frac{x'}{w'} \quad y = \frac{y'}{w'}$$

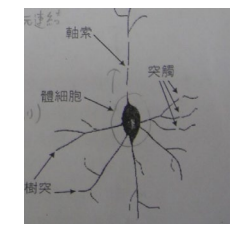
$$p = M_{\text{int}} \cdot P_c$$

B1-3 DL Model - $AX=b$: Back Propagation (4/3)

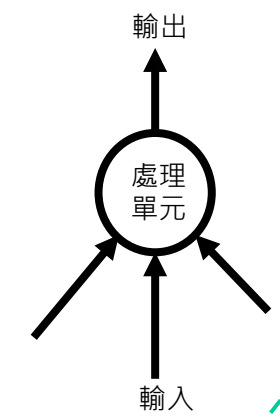
寫程式不會以equation的方式，會以vector，matrix ($Ax = b$), homogenous matrix, 的方式撰寫。

- For the input, why are extraction features better than raw image??

Review NN (Ch1)



生物神經元模型



人工神經元模型

$y_j = f(\sum_i W_{ij} x_i - \theta_j)$

constant

bias

A neuron unit

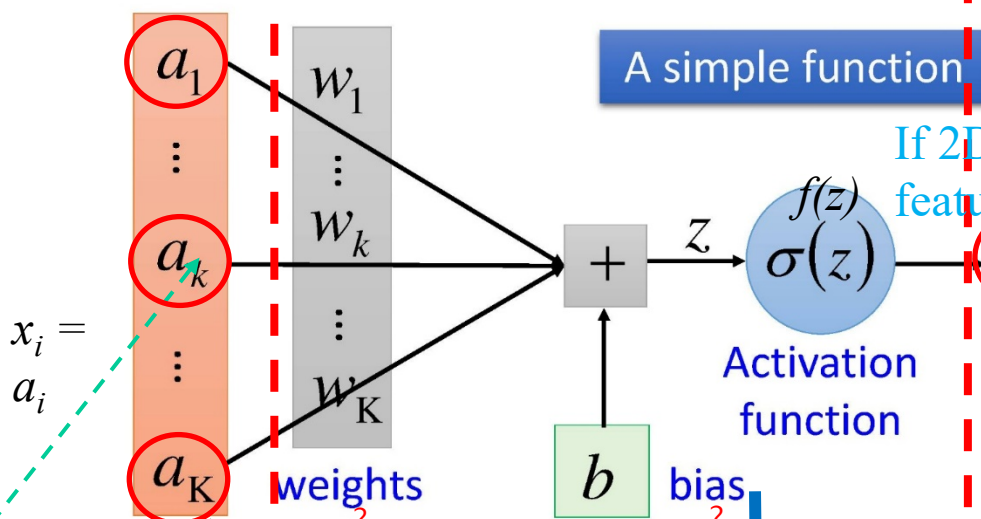
Neural Network

One **Neuron**

Linear Combination:

$z = a_1 w_1 + \dots + a_k w_k + \dots + a_K w_K + b$

Constant, bias



input : (20x20個pixel的圖)
 $x_1, x_2, \dots, x_n \approx 100$

Linear Discrimination by Linear Combination

-One weak classifier is one haar filter and is also as one neuron unit
-One strong classifier is as one layer

Linear Combination:

$W a = z$

filter 1 $\begin{bmatrix} w_{11} & w_{21} & \dots & w_{k1} & \dots & w_{N1} \end{bmatrix} b_1$

filter j $\begin{bmatrix} w_{12} & w_{22} & \dots & w_{k2} & \dots & w_{N2} \end{bmatrix} b_2$

$\vdots$

$\begin{bmatrix} w_{1j} & w_{2j} & \dots & w_{kj} & \dots & w_{Nj} \end{bmatrix} b_j$

$\vdots$

$\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_k \\ \vdots \\ a_N \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_k \\ \vdots \\ z_N \end{bmatrix}$

$(N+1) \times 1$

I/P **O/P**

J input data vectors (here J=1)
1 J-dim output result vector, here J=1)

A simple function

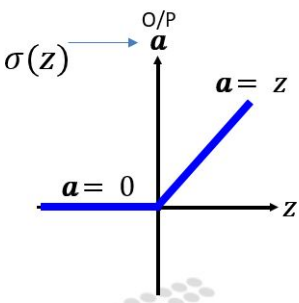
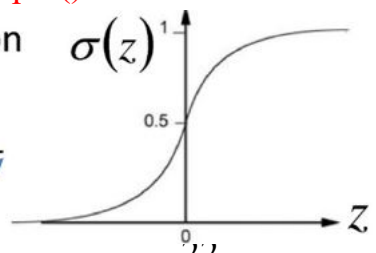
If 2D, one filter creates one feature map by convolution

$a = \sigma(z)$
Here, the output is a , which is one input element for next neuron
If input is 2D map, output will be J 2D feature maps for J 2D filter

Non-Linear Discrimination by Kernel Function / Activation Function as SVM $\phi()$

Sigmoid Function

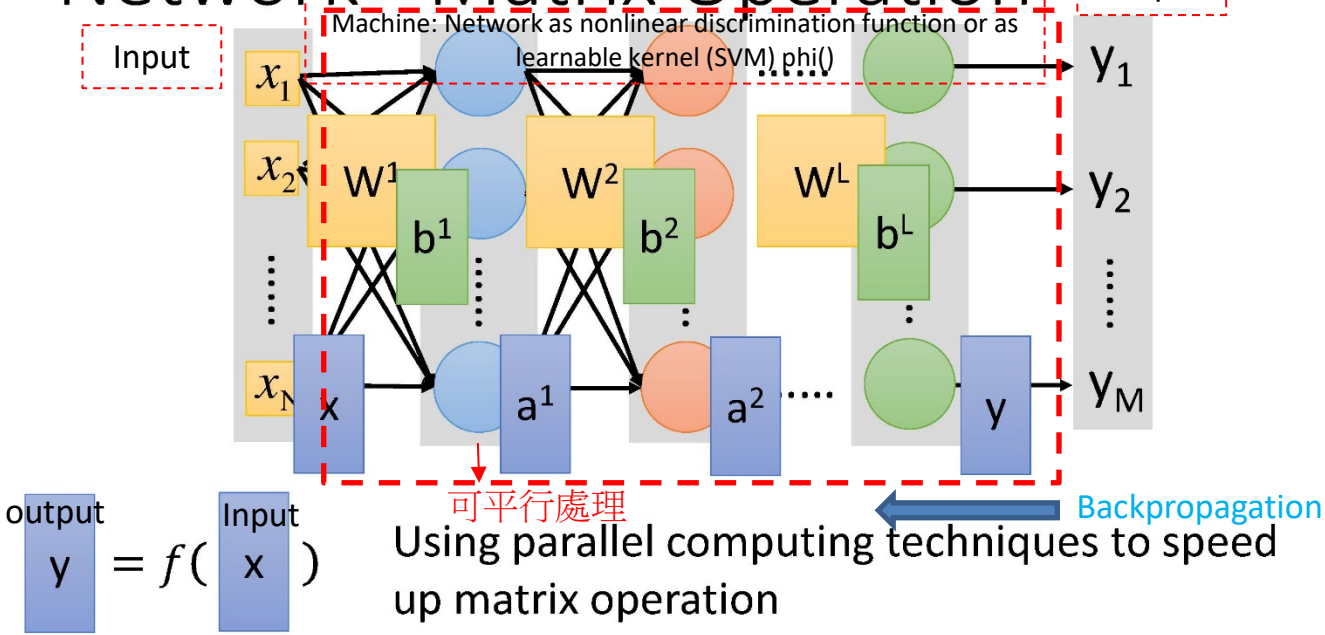
$\sigma(z) = \frac{1}{1 + e^{-z}}$



B1-3 DL Model - $AX=b$: Back Propagation (4/4)

Ground truth y : 0,1,2,3,...,5,...,8,9
 Label: 0,0,1,0,...,0,...,0,0
 Prediction: $0 \leq y' \leq 1$

Fully Connect Feedforward Network - Matrix Operation



$$\begin{bmatrix} w_{11} & w_{21} & \dots & w_{k1} & \dots & w_{N1} & b_1 \\ w_{12} & w_{22} & \dots & w_{k2} & \dots & w_{N2} & b_2 \\ \vdots & \vdots & & \vdots & & \vdots & \vdots \\ w_{1j} & w_{2j} & \dots & w_{kj} & \dots & w_{Nj} & b_j \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_k \\ \vdots \\ x_N \\ 1 \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_k \\ \vdots \\ z_j \end{bmatrix}$$

Label vector

$y=a, x=z$
 $a=\sigma(z)$

Replace $a(L-1)$ para.
 $\Rightarrow$ As forward...

$$= \sigma \left(W^L \dots \sigma \left(W^2 \sigma \left(W^1 x + b^1 \right) + b^2 \right) \dots + b^L \right)$$

$L=20$ layers

Labeling vector

$W_L X_L = Y$

$[W_L * W_{L-1}] X_{L-1} = Y$

$[W_L * \dots * W_1] X_1 = Y$

$W?$ I/P & O/P Known

Solve W by gradient decent



B1-3 Optimization of Homogenous Matrix: $Ax=b$

■ Solution/Optimization of Homogenous Matrix

-Local optimization (0 moment): 1.1 and 1.2 =>

-Global optimization: 2 => 2.1) LM 2.2) EM 2.3) SGD (Stochastic Gradient Decent)

1.1 Closed-Form Solution:

$Ax = 0 \Rightarrow A^t A = \text{Covariance Matrix} = \text{SVD} = U W U^T$: Smallest eigenvalue $> 0 \rightarrow$ eigenvector

1.2 Pseudo Inverse:

$$Ax = b, x = (A^T A)^{-1} * A^T b$$

2.0 Sum of Squared Difference: (max likelihood – exponential term)

Iteration $\min E = \sum [Ax - b]^2$

- Why does deep learning apply Mean Squared Error
- $\min E = 1/B * \sum_1^B [Ax - b]^2$ where B is the batch size

2.1 $Ax = b'$: estimation value. b : ground truth, $E = \sum [b' - b]^2$

- Initial value estimation => Pseudo Inverse (linear approach)
- L-M (Levenberg-Marquardt Algorithm: non-linear approach)
 - First order Taylor series expansion

b.2 2nd order Taylor series expansion (sensitive to noise)

2.2 $Ax = b'$: estimation value. b'' : estimation value, $E = \sum [b' - b'']^2$

- Unsupervised Learning Use at Reinforcement learning for reward r

Machine learning for prediction a. EM (Expected-Maximization), initial $b' =$ average value

2.3 Stochastic Gradient Descent

- Deep Learning: Stochastic Gradient Decent

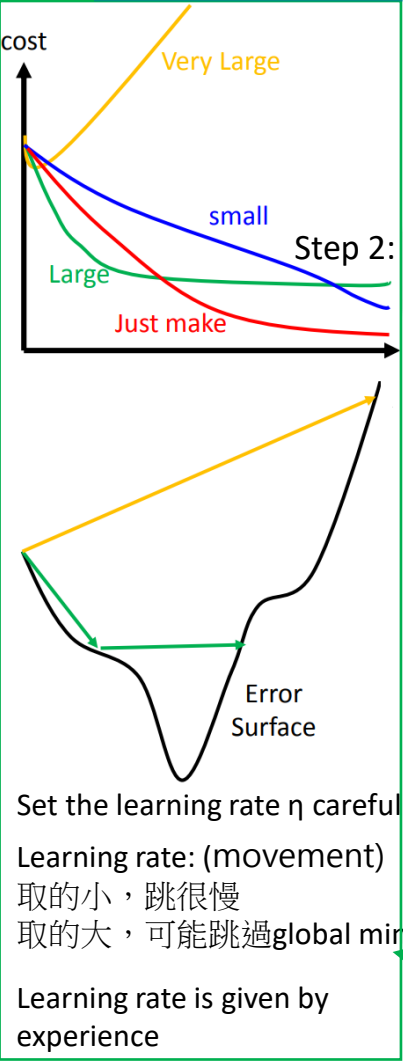
3. Lagrange Approach (outlier) with constraint

$$\min E = \sum [Ax - b]^2 + \lambda (x^2 + y^2)^2$$

- Constraint: GAN ...

B1-3.1 Gradient Descent

Reference:[http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/DNN%20\(v4\).pdf](http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/DNN%20(v4).pdf)



Gradient Descent

Network parameters $\theta = \{w_1, w_2, \dots, b_1, b_2, \dots\}$

Learning Target

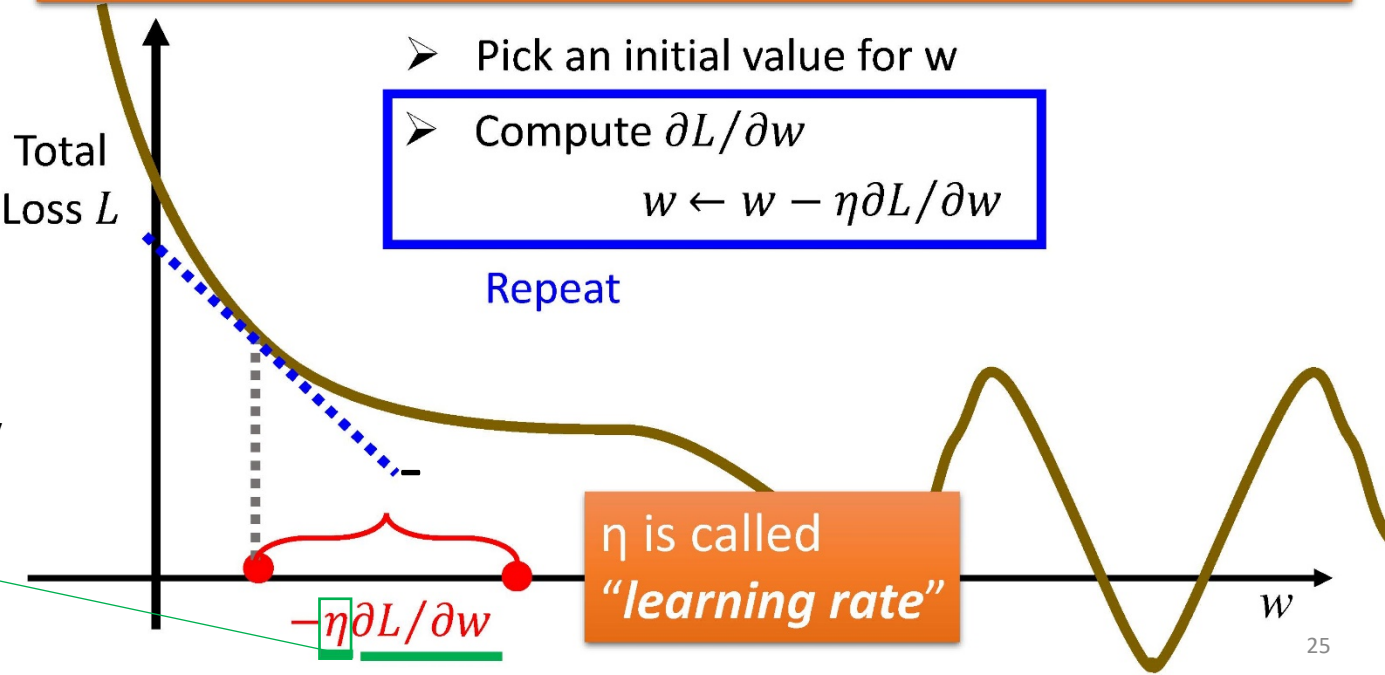
Find the network parameters θ^* that minimize total loss

➤ Pick an initial value for w

➤ Compute $\partial L / \partial w$

$$w \leftarrow w - \eta \partial L / \partial w$$

Repeat



B1-3.2 Lagrange multiplier method – Metrics

Using Ridge Regression to solve t:

Lambda called Lagrange Multiplier

岭回归(Ridge Regression)是在平方误差的基础上增加正则项

$$\text{Loss } L = \sum_{i=1}^n \left(y_i - \sum_{j=0}^p w_j x_{ij} \right)^2 + \lambda \sum_{j=0}^p w_j^2$$

With $\lambda=1000$

J: This is for bias optimization??

J: This is for variance optimization

So this constraint term is for variance??

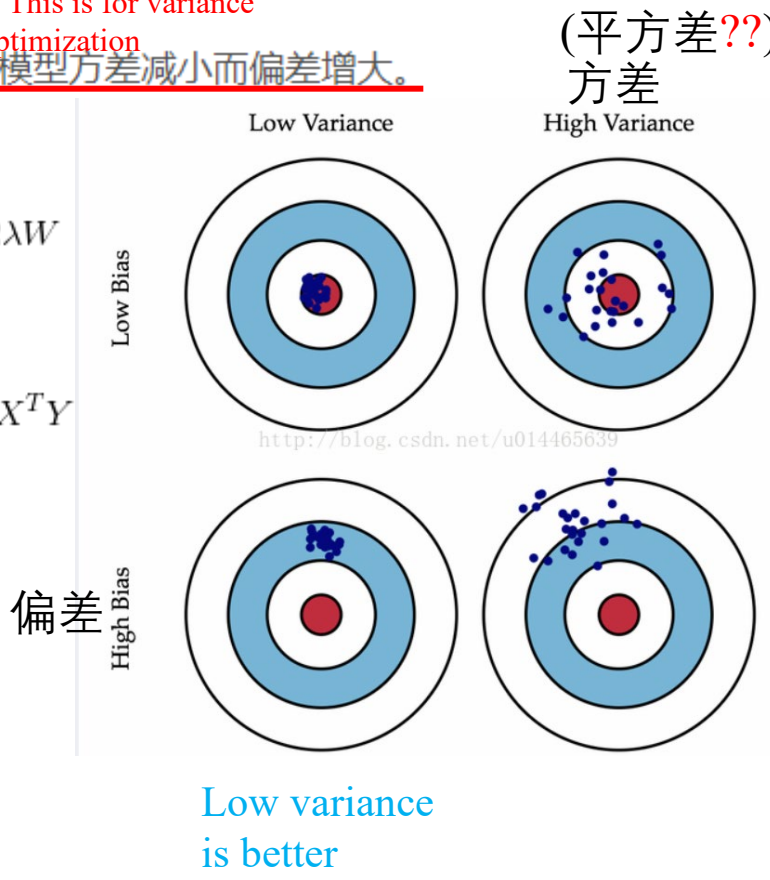
通过确定λ的值可以使得在方差和偏差之间达到平衡：随着λ的增大，模型方差减小而偏差增大。

对w求导，结果为

$$\frac{\partial L}{\partial w} = 2X^T(Y - XW) - 2\lambda W$$

令其为0，可求得w的值：

$$\hat{w} = (X^T X + \lambda I)^{-1} X^T Y$$



3.1 Low-Pass Filter for Denoise and High-Pass Filter for Edge Detection (Feature Extraction) via Convolution → CNN



3.1-0 Diffusion Model

Denoising Diffusion Probabilistic Models

NIPS 2020

Jonathan Ho

UC Berkeley

jonathanho@berkeley.edu

Ajay Jain

UC Berkeley

ajayj@berkeley.edu

Pieter Abbeel

UC Berkeley

pabbeel@cs.berkeley.edu

Student: MS2023 盧識永 V1

Advisor: Jenn-Jier James Lien

Robotics Lab, CSIE NCKU

Edited date: 2024/09/23

3.1-0 Diffusion Model - Introduction: Motivation and Objective

1. Motivation:

- 1) Deep generative models of all kinds have recently exhibited high quality samples in a wide variety of data modalities.
 - 1.1) Generative adversarial networks (GANs), autoregressive models, flows, and variational autoencoders (VAEs) have synthesized striking image and audio samples [14, 27, 3, 58, 38, 25, 10, 32, 44, 57, 26, 33, 45].
 - 1.2) There have been remarkable advances in energy-based modeling and score matching that have produced images comparable to those of GANs [11, 55].

2. Objective:

straightforward to define and efficient to train

- 1) Present progress in **Diffusion Probabilistic Models**.
 - 1.1) A class of latent variable models inspired by considerations from nonequilibrium thermodynamics. 某個大於等於1的時刻 t ，其隨機變量只會與前一個時刻之間有條件分布的關係，並不會依賴更久之前的隨機變量。
 - 1.2) A parameterized **Markov chain** trained using variational inference to produce samples matching the data after finite time. $P(x_t) = P(x_t|x_{t-1})P(x_{t-1}|x_{t-2}) \dots \cong P(x_t|x_{t-1})P(x_{t-1})$ (causal)
- 2) Present a more refined analysis of this phenomenon in the language of lossy compression, and we show that the sampling procedure of diffusion models is a type of progressive decoding.
 - ➔ Resembles autoregressive decoding along a bit ordering that vastly generalizes what is normally possible with autoregressive models.

Generated by Denoising Diffusion
Probabilistic Models (DDPM)



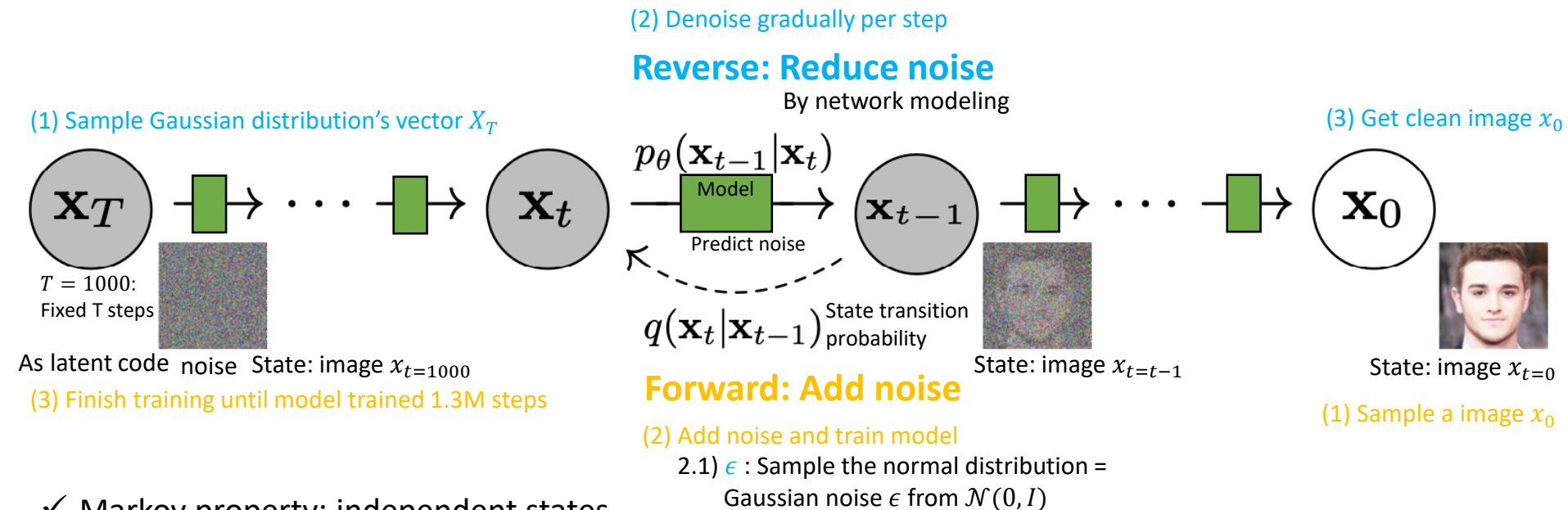
3.1-0 Diffusion Model - Introduction: Global Framework - Concept

3. Global Framework

Q: Datasets distribution 很複雜，無法用數學模型表示，意味無法做好 generation task.

A: 1) **Forward Process:** Step by step, 從複雜的 distribution (image) 變成簡單好分析的 distribution (noise). (orange steps)

2) **Reverse Process:** 反之，從 noise 還原原始數據的樣本。(blue steps)



✓ Markov property: independent states.

✓ Forward & reverse process (狀態前後變換($x_t \rightarrow x_{t-1}$)) can be approximated by Gaussian distribution if the step is small enough. → 可被數學工具分析。

3.1-0 Diffusion Model - Experimental Result

■ Progressive Generation of Unconditional CIFAR10

progressive decompression from random bits.



Figure 6: Unconditional CIFAR10 progressive generation ($\hat{x}_0$ over time, from left to right). Extended samples and sample quality metrics over time in the appendix (Figs. 10 and 14).

■ Interpolation of CelebA-HQ 256x256

- 1) $x'_0 \sim q(x_0)$
- 2) $x'_t \sim q(x_t | x_0)$ fixed the noise for different values of λ so x_t and x'_t remain the same
- 3) Decoding the linearly interpolated latent $\bar{x}_t = (1 - \lambda)x_0 + \lambda x'_0$ into image space by the reverse process, $\bar{x}_0 \sim p(x_0 | \bar{x}_t)$.
 ✓ Using the reverse process to remove artifacts from linearly interpolating corrupted versions of the source images.

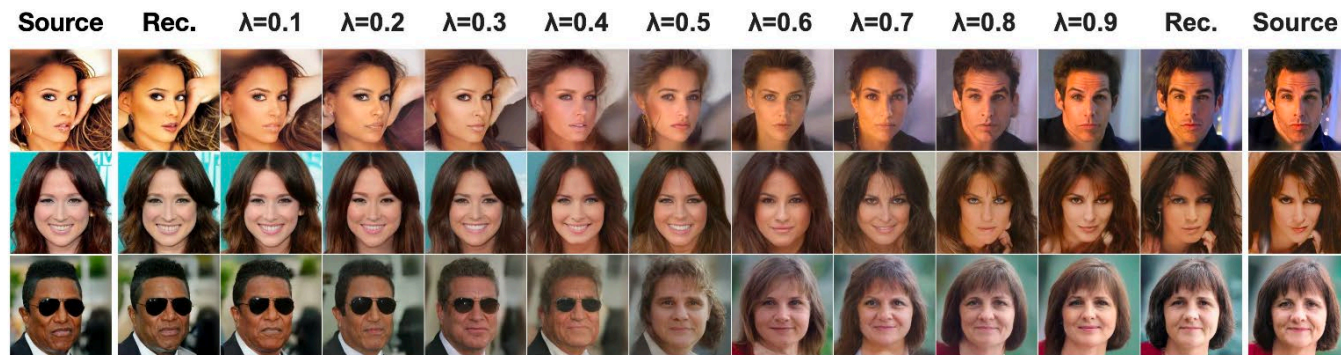
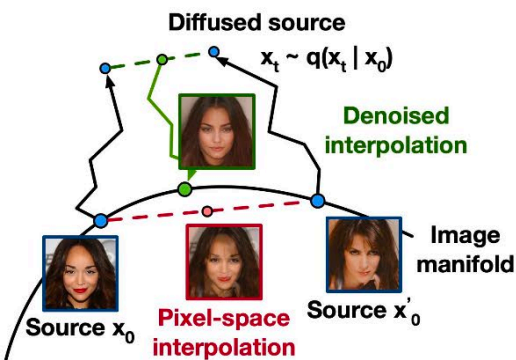


Figure 8: Interpolations of CelebA-HQ 256x256 images with 500 timesteps of diffusion.

3.1-1 Two Noise Types (1/2)

◆ 2 Noise Types:

1) Gaussian noise

- A Gaussian distribution of noise samples.
- Using Gaussian filter

2) Impulse noise – Salt and pepper noise

- It represents itself as randomly occurring white and black pixels.
- An effective noise reduction method for this type of noise involves the usage of a median filter.

◆ We focus on reducing the Gaussian noise and impulse (salt and pepper) noise.



3.1-1 Two Noise Types (2/2)



Noise free



1) With Gaussian noise



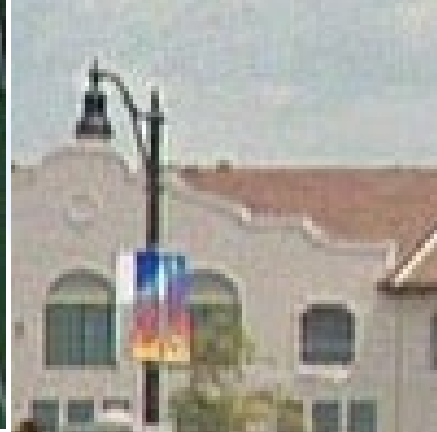
2) With impulse noise



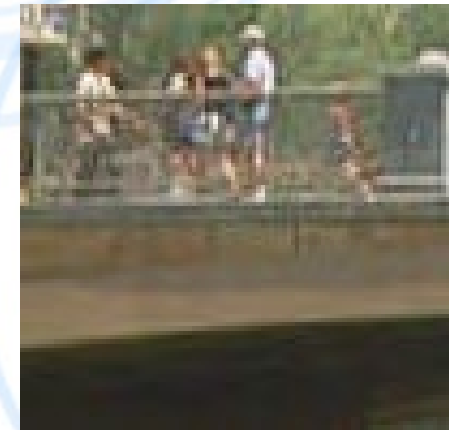
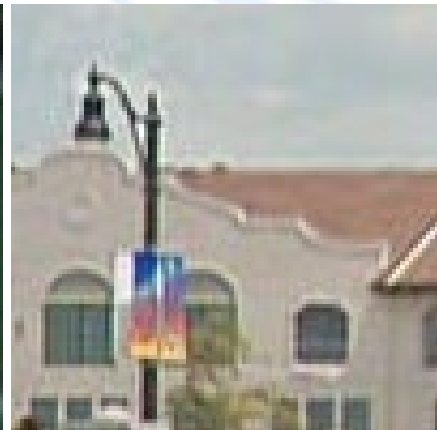
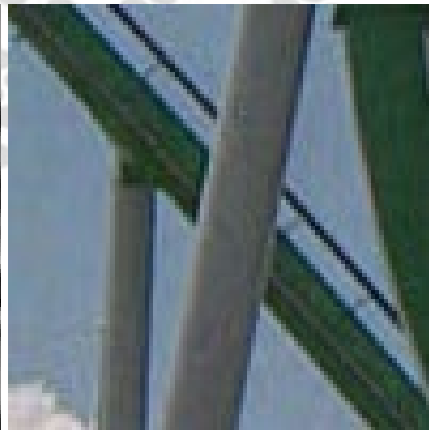
3.1-1 Noises in Spatial and Temporal Domains

- ◆ Denoise: 2D in Spatial Domain + 1D in Temporal Domain
- ◆ Noise is filtered without losing edge information.

Before
NR



After
NR



3.1-2 Low-Pass Smoothing Filters: Goal - Denoise

◆ 4 Types of Low-Pass Smoothing Filters to Denoise

Goal

In this tutorial you will learn how to apply diverse linear filters to smooth images using OpenCV functions such as:

- 1) • `blur` `average`
- 2) • `GaussianBlur`
- 3) • `medianBlur`
- 4) • `bilateralFilter`

3.1-2 Low-Pass Smoothing Filters: Theory - Convolution

Theory

Note: The explanation below belongs to the book *Computer Vision: Algorithms and Applications* by Richard Szeliski and to *LearningOpenCV*

- Smoothing, also called blurring, is a simple and frequently used image processing operation.
- There are many reasons for smoothing. In this tutorial we will focus on smoothing in order to reduce noise (other uses will be seen in the following tutorials).
- To perform a smoothing operation we will apply a *filter* to our image. The most common type of filters are *linear*, in which an output pixel's value (i.e. $g(i, j)$) is determined as a weighted sum of input pixel values (i.e. $f(i + k, j + l)$):

$$g(i, j) = \sum_{k, l} f(i + k, j + l) h(k, l)$$

$$g[x, y] = \sum_{u=-\frac{N}{2}}^{\frac{N}{2}-1} \sum_{v=-\frac{N}{2}}^{\frac{N}{2}-1} h[u, v] \cdot f(x + u, y + v)$$

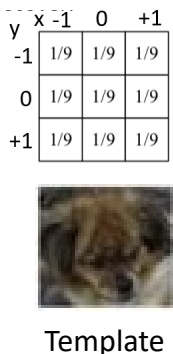
$N=3$, window size = radius = 3x3

$h(k, l)$ is called the kernel, which is nothing more than the coefficients of the filter

It helps to visualize a filter as a window of coefficients sliding across the image.

→ threshold

- There are many kind of filters, here we will mention the most used:



Filter or template



Compare the template image against the source image by sliding it.

3.1-2 Filters: Low-Pass Vs. High-Pass

1. Low-pass filter: Smoothing filter to denoise

- 低通濾波一般又稱為平滑濾波(Smoothing filter)，通常用於模糊化與減少高頻雜訊。
。最常見的此類濾波有平均濾波(Averaging filter)與高斯平滑濾波 (Gaussian filter)。
- Original energy, such as sum of each pixel gray value $I^2(x,y)$ over entire image, will change. So it is hard to recover.

$\sin\theta$

$\cos\theta$

θ

	x -1	0	+1
y -1	1/9	1/9	1/9
0	1/9	1/9	1/9
+1	1/9	1/9	1/9

$h[u,v]$
3x3 (a) 平均濾波

1/16	2/16	1/16
2/16	4/16	2/16
1/16	2/16	1/16

$h[u,v]$
3x3 (b) 高斯平滑濾波

Sum of filter
coefficients = 1

2. High-pass filter: Extract high frequency texture

- 高通濾波器是用來突顯影像中高頻特徵 feature 的成分，在影像較為模糊的情況可讓影像較為清晰。

-1/9	-1/9	-1/9
-1/9	8/9	-1/9
-1/9	-1/9	-1/9

$h[u,v]$
3x3 高通濾波係數

Sum of filter
coefficients = 0

→ threshold

3.1-2 Step 1 - Convolutional neural networks for isolated character recognition

Convolutional layer

Padding = 0 pixel
Stride = Shift = 1 pixel

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

3x3 Filter coefficients

1	0	1
0	1	0
1	0	1

$$\begin{aligned} &1 \times 1 + 1 \times 0 + 1 \times 1 + \\ &0 \times 0 + 1 \times 1 + 1 \times 0 + \\ &0 \times 1 + 0 \times 0 + 1 \times 1 \\ &= 4 \end{aligned}$$

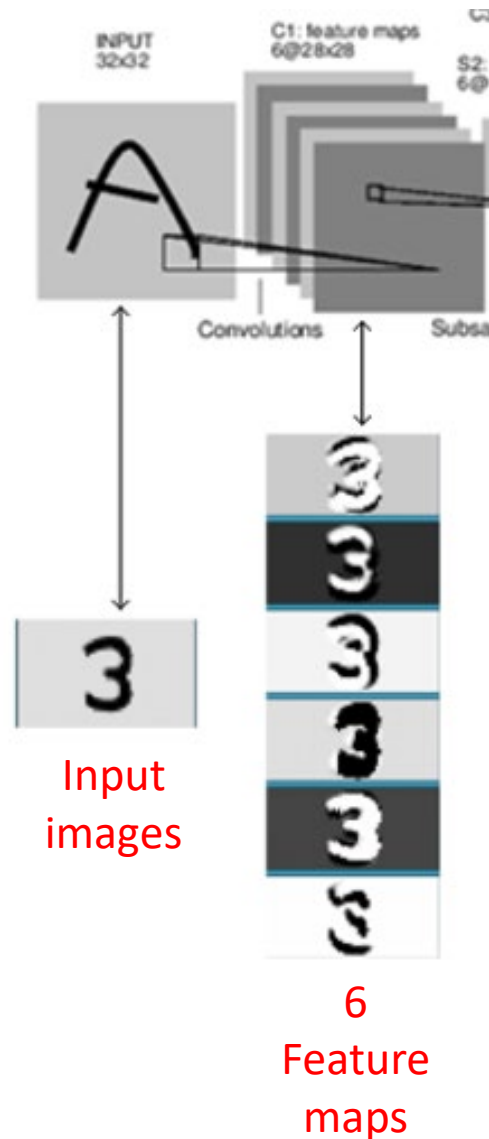
4		

Convolved
Feature

4	3	4
2	4	3
2		

$$\begin{aligned} &0 \times 1 + 0 \times 0 + 1 \times 1 + \\ &0 \times 0 + 0 \times 1 + 1 \times 0 + \\ &0 \times 1 + 1 \times 0 + 1 \times 1 \\ &= 2 \end{aligned}$$

Convolved
Feature



How convolutional layer works and its result

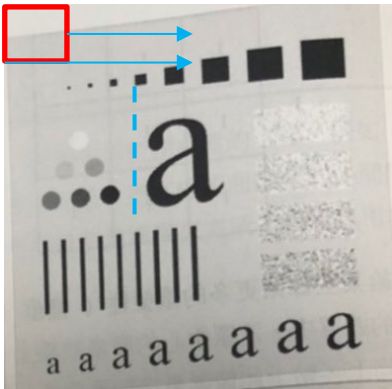
3.1-2.1 Low-Pass Smoothing Filter: Average Filter

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

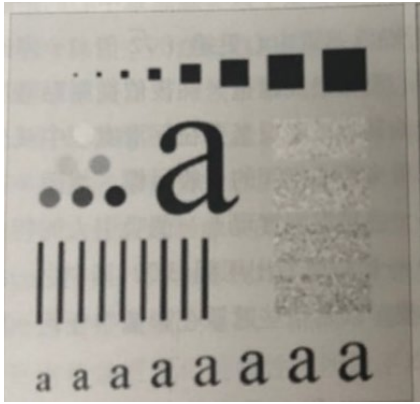
(a)平均濾波



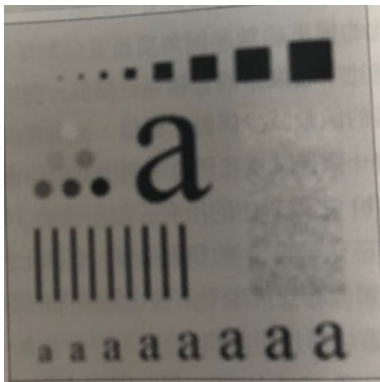
(a) 原始影像



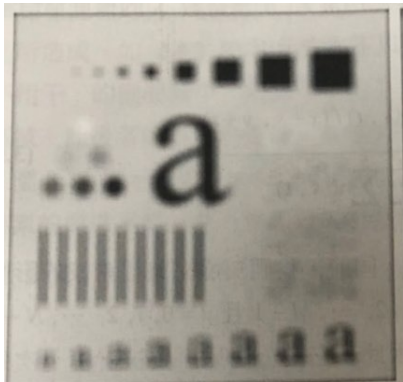
(b)



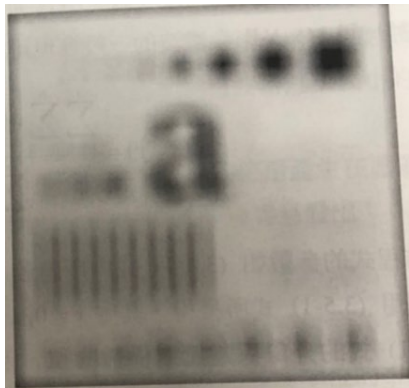
(c)



(d)



(e)



(f)

(b)~(f)分別是以 $m=3, 5, 9, 15$ 以及 35 的方形平均濾波的平滑結果。
Size $m \times m$

3.1-2.2 Low-Pass Smoothing Filter: Gaussian Filter

Gaussian Filter

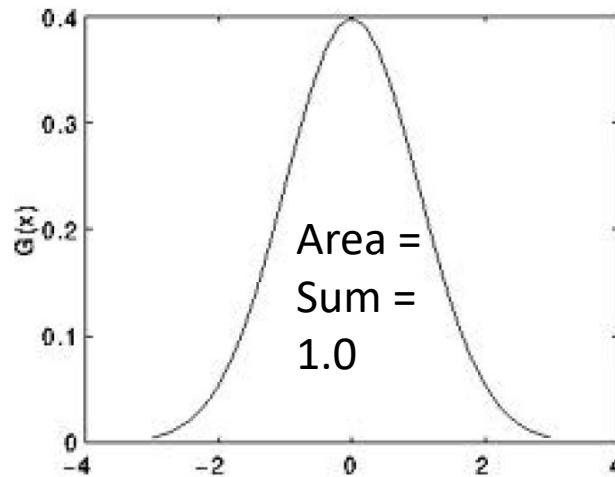
- Probably the most useful filter (although not the fastest). Gaussian filtering is done by convolving each point in the input array with a Gaussian kernel and then summing them all to produce the output array.
- Just to make the picture clearer, remember how a 1D Gaussian kernel look like?

e: Activation function

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



Normalization term: to between 0.0 ~ 1.0



近似3*3 Gaussian Filter

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

generalized weighted smoothing filter

Assuming that an image is 1D, you can notice that the pixel located in the middle would have the biggest weight. The weight of its neighbors decreases as the spatial distance between them and the center pixel increases.

Note: Remember that a 2D Gaussian can be represented as :

$$G_0(x, y) = A e^{-\frac{(x-\mu_x)^2}{2\sigma_x^2} - \frac{(y-\mu_y)^2}{2\sigma_y^2}}$$

$$\frac{1}{\sigma\sqrt{2\pi}} \times \frac{1}{\sigma\sqrt{2\pi}}$$

$$A = \frac{1}{2\pi\sigma^2}$$

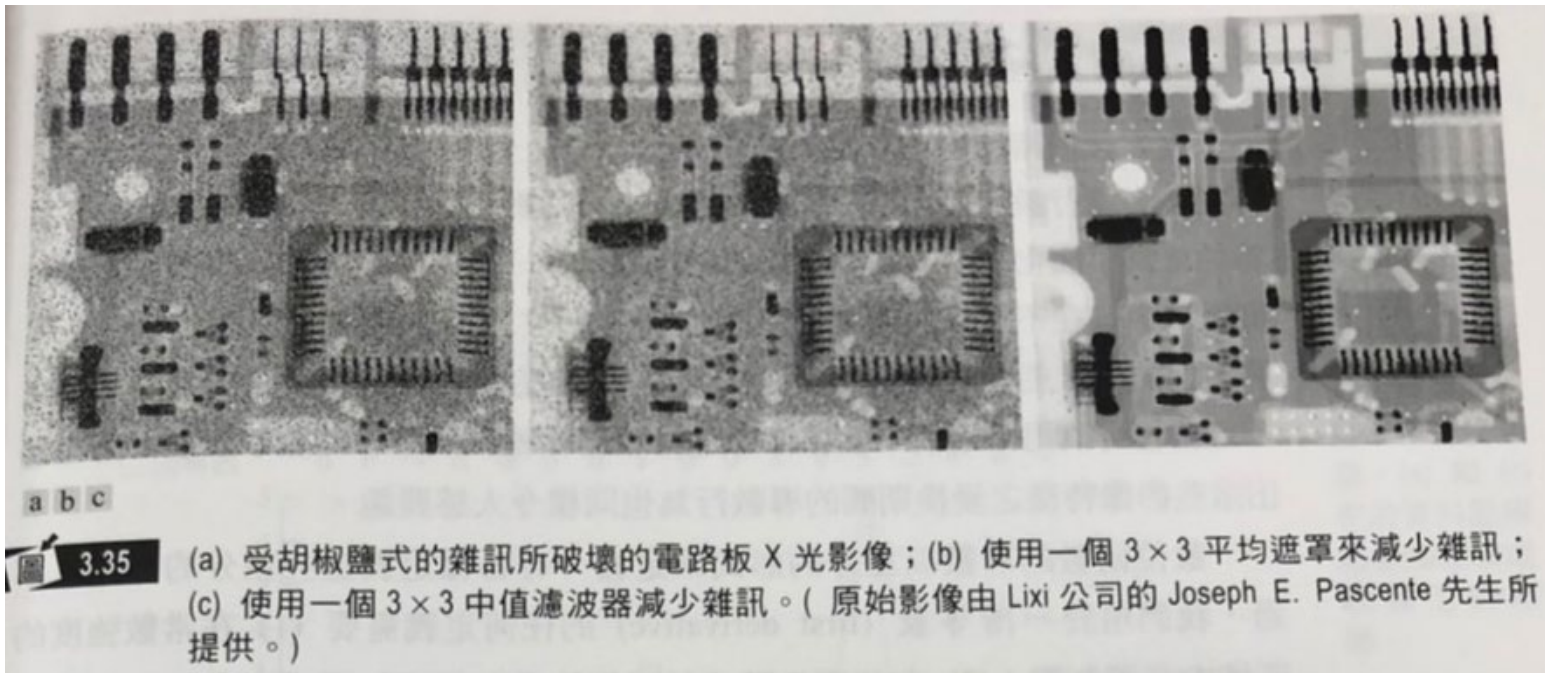
where μ is the mean (the peak) and σ represents the ~~variance~~ **Standard deviation** (per each of the variables x and y)

3.1-2.3 Low-Pass Smoothing Filter: Median Filter

Median Filter

The median filter run through each element of the signal (in this case the image) and replace each pixel with the median of its neighboring pixels (located in a square neighborhood around the evaluated pixel).

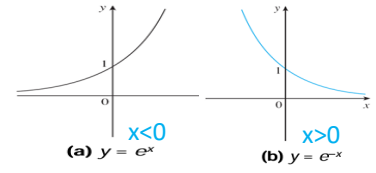
Median filter: 經常用於去除圖像或者其他信號中的雜訊，中值濾波器在脈衝雜訊（impulse）出現時，特別有用，因為脈衝雜訊看起來像是疊加在影像上的白點和黑點，所以又稱為胡椒鹽式雜訊（salt-and-pepper noise）。



3.1-2.3 Low-Pass Smoothing Filter: Bilateral Filter

A bilateral filter

- is a non-linear, edge-preserving, and noise-reducing smoothing filter for images.
- It replaces the intensity of each pixel with a weighted average of intensity values from nearby pixels. This weight can be based on a Gaussian distribution.



Bilateral Filter

- So far, we have explained some filters which main goal is to *smooth* an input image. However, sometimes the filters do not only dissolve the noise, but also smooth away the edges. To avoid this (at certain extent at least), we can use a bilateral filter.
- In an analogous way as the Gaussian filter, the bilateral filter also considers the neighboring pixels with weights assigned to each of them. These weights have two components, the first of which is the same weighting used by the Gaussian filter. The second component takes into account the difference in intensity between the neighboring pixels and the evaluated one.
- http://homepages.inf.ed.ac.uk/rbf/CVonline/LOCAL_COPIES/MANDUCHI1/Bilateral_Filtering.html
- For a more detailed explanation you can check -----

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma_s}(\|p - q\|) G_{\sigma_r}(\|I_p - I_q\|) I_q$$

Annotations for the formula:

- $\frac{1}{W_p}$: Normalization Factor
- $G_{\sigma_s}(\|p - q\|)$: Space Weight (Distance)
- $G_{\sigma_r}(\|I_p - I_q\|)$: Edge, gradient (Range Weight)

Hi gradient, low weight/affect

Distance: Closer center (i,j)

Edge, gradient

3.1-3 High-Pass Filter: Feature Extraction - Edge Detection (1/3)

Theory

Note: The explanation below belongs to the book **Learning OpenCV** by Bradski and Kaehler.

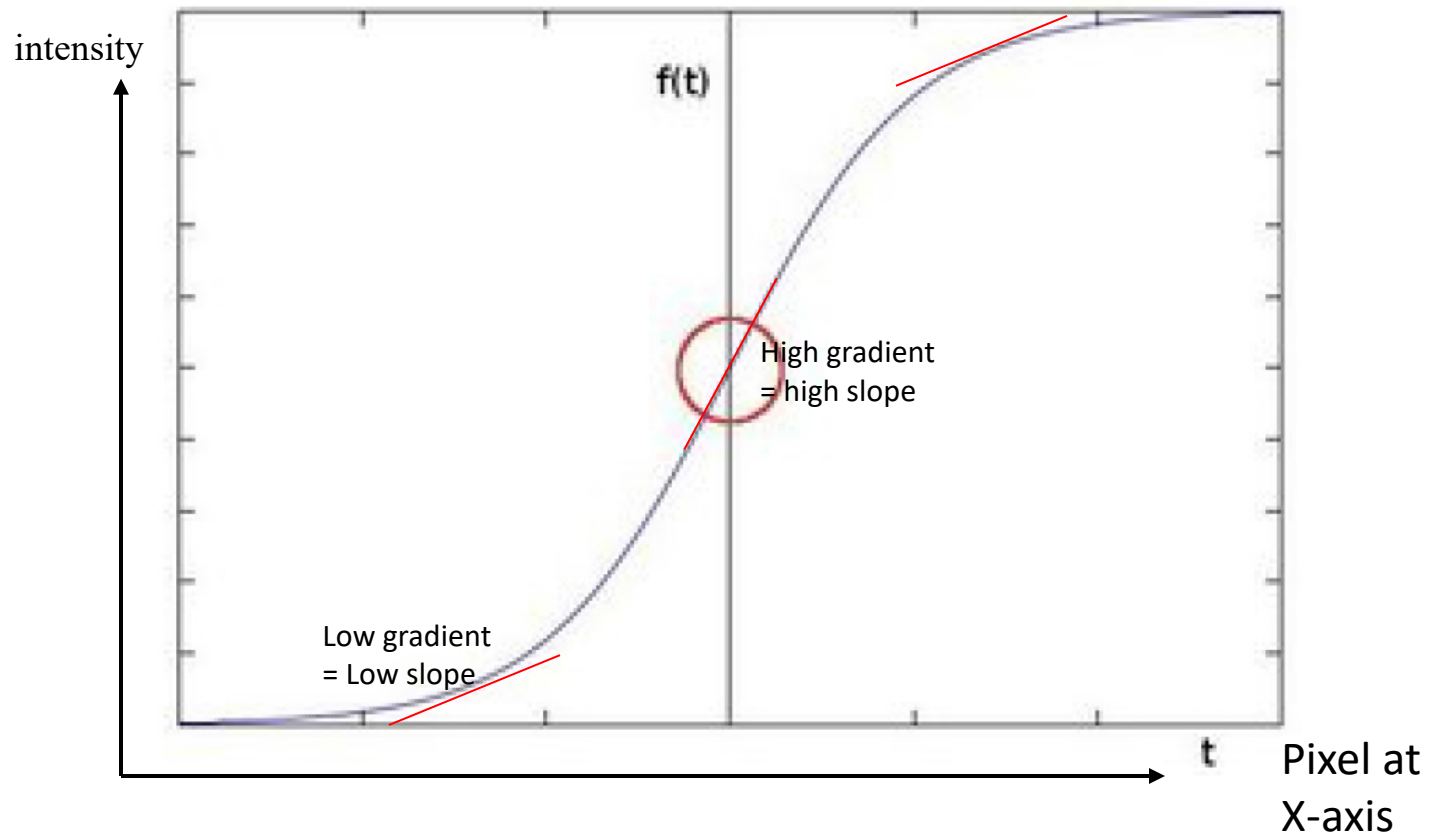
1. In the last two tutorials we have seen applicative examples of convolutions. One of the most important convolutions is the computation of derivatives in an image (or an approximation to them).
2. Why may be important the calculus of the derivatives in an image? Let's imagine we want to detect the *edges* present in the image. For instance:



You can easily notice that in an edge, the pixel intensity *changes* in a notorious way. A good way to express changes is by using *derivatives* A high change in gradient indicates a major change in the image.

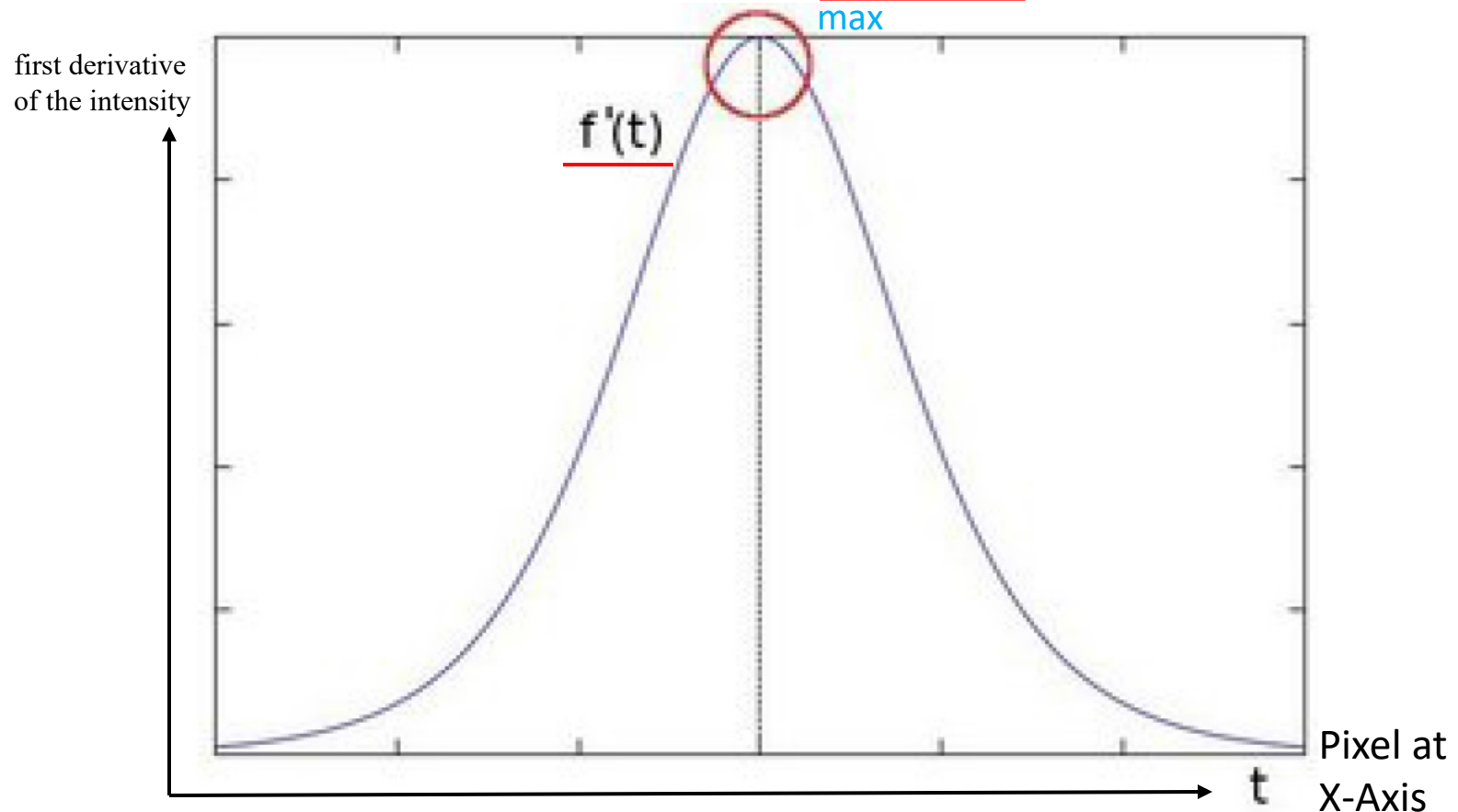
3.1-3 High-Pass Filter: Feature Extraction - Edge Detection (2/3)

3. To be more graphical, let's assume we have a 1D-image. An edge is shown by the “jump” in intensity in the plot below:



3.1-3 High-Pass Filter: Feature Extraction - Edge Detection (3/3)

4. The edge “jump” can be seen more easily if we take the first derivative (actually, here appears as a maximum)



5. So, from the explanation above, we can deduce that a method to detect edges in an image can be performed by locating pixel locations where the gradient is higher than its neighbors (or to generalize, higher than a threshold).
6. More detailed explanation, please refer to **Learning OpenCV** by Bradski and Kaehler

3.1-3 High-Pass Filter: Feature Extraction - Edge Detection

- ◆ Making your own linear filters!:
Spatial Correlation and Convolution

Using correlation or convolution to perform spatial filtering is a matter of preference. In fact, because either Eq. (3.4-1) or (3.4-2) can be made to perform the function of the other by a simple rotation of the filter, what is important is that the filter mask used in a given filtering task be specified in a way that corresponds to the intended operation. All the linear spatial filtering results in this chapter are based on Eq. (3.4-1).

Finally, we point out that you are likely to encounter the terms, convolution filter, convolution mask or convolution kernel in the image processing literature. As a rule, these terms are used to denote a spatial filter, and not necessarily that the filter will be used for true convolution. Similarly, “convolving a mask with an image” often is used to denote the sliding, sum-of-products process we just explained, and does not necessarily differentiate between correlation and convolution. Rather, it is used generically to denote either of the two operations. This imprecise terminology is a frequent source of confusion.

3.1-3.1 High-Pass Filter: Sobel Filter (1/4)

Sobel Operator

1. The Sobel Operator is a discrete differentiation operator. It computes an approximation of the gradient of an image intensity function.
2. The Sobel Operator combines Gaussian smoothing and differentiation.

```
cvSobel(  
    const CvArr* src,  
    CvArr*      dst,  
    int         xorder, 0, 1, 2  
    int         yorder, 0, 1, 2  
    int         aperture_size = 3  
);
```

derivative
= 3x3

Here, src and dst are your image input and of the derivative. Typically you'll use 0, 1, 2

$$\left\{ \begin{array}{l} \text{Magnitude} \quad M = \sqrt{I_x^2 + I_y^2} \\ \text{Direction} \quad \theta = \arctan^2 \frac{I_y}{I_x} \end{array} \right.$$

3.1-3.1 High-Pass Filter: Sobel Filter (2/4)

$$\frac{\partial I}{\partial x} = \lim_{\Delta x \rightarrow 0} \frac{I(x+\Delta x) - I(x-\Delta x)}{2\Delta x}$$

$\frac{I(x+1) - I(x-1)}{2}$

$\begin{bmatrix} -1 & 0 & +1 \\ -2 & 1 & 2 \\ +1 & 0 & -1 \end{bmatrix}$

Sobel

$$D_x = \begin{bmatrix} 1 & 0 & -1 \\ -2 & 0 & 2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

inverted

$$= \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

weight

$$D_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

output, and xorder and yorder are the orders

at most 2; a 0 value indicates no derivative

$$I(x,y) * D_x = I_x(x,y)$$

$$I(x,y) * D_y = I_y(x,y)$$

3.1-3.1 High-Pass Filter: Sobel Filter (3/4)

Formulation

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

Assuming that the image to be operated is I:

1. We calculate two derivatives:

- (a) **Horizontal changes:** This is computed by convolving I with a kernel $\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$ with odd size. For example for a kernel size of 3, $\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$ would be computed as:

$$Dy(x, y) = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * I$$

- (b) **Vertical changes:** This is computed by convolving I with a kernel $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$ with odd size. For example for a kernel size of 3, $\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$ would be computed as:

$$Dx(x, y) = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * I$$

2. At each point of the image we calculate an approximation of the *gradient* in that point by combining both results above:

$$G = \sqrt{G_x^2 + G_y^2}$$

slow

Although sometimes the following simpler equation is used:

$$G = |G_x| + |G_y|$$

fast

3.1-3.1 High-Pass Filter: Sobel Filter (4/4)

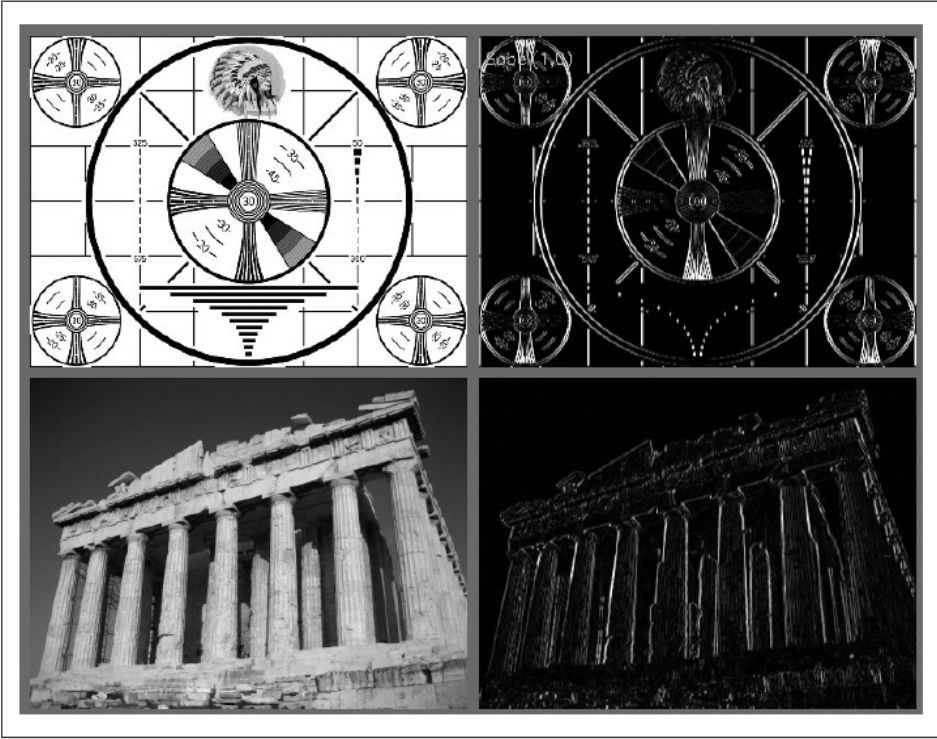


Figure 6-3. The effect of the Sobel operator when used to approximate a first derivative in the x-dimension

$$Dy(x, y) = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

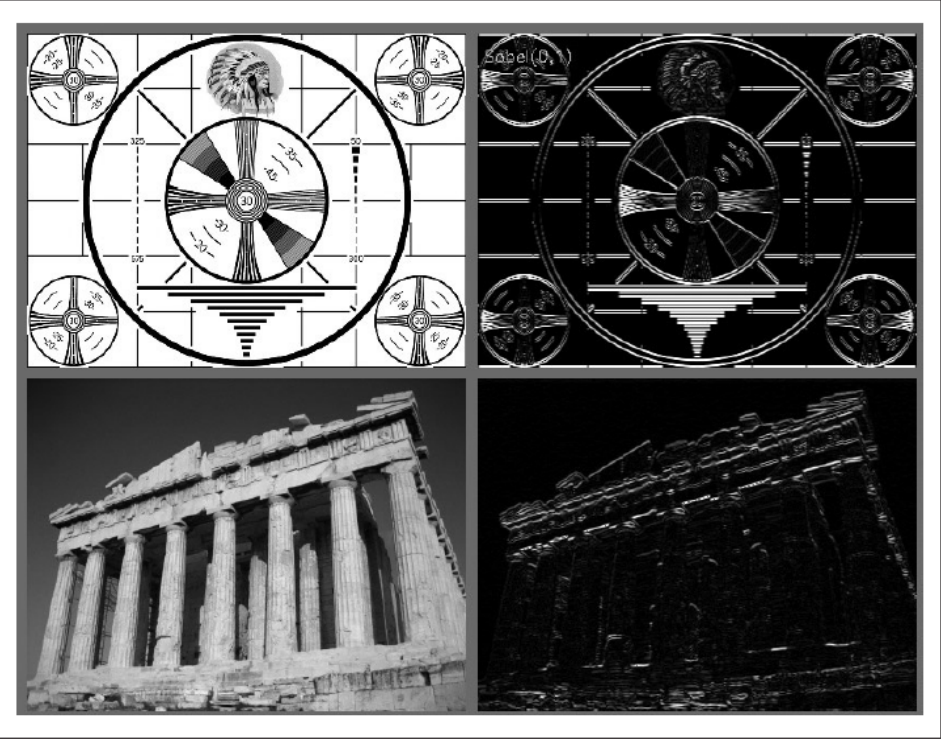


Figure 6-4. The effect of the Sobel operator when used to approximate a first derivative in the y-dimension

$$Dx(x, y) = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

3.1-3.2 High-Pass Filter: Scharr Filter

Scharr:

Note:

When the size of the kernel is 3, the Sobel kernel shown above may produce noticeable inaccuracies (after all, Sobel is only an approximation of the derivative). OpenCV addresses this inaccuracy for kernels of size 3 by using the Scharr function. This is as fast but more accurate than the standar Sobel function. It implements the following kernels:

$$\begin{aligned} Dy(x, y) &= : \begin{bmatrix} -3 & 0 & +3 \\ -10 & 0 & +10 \\ -3 & 0 & +3 \end{bmatrix} \\ Dx(x, y) &= : \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ +3 & +10 & +3 \end{bmatrix} \end{aligned}$$

J: Enhance slope

You can check out more information of this function in the OpenCV reference ([Scharr](#)). Also, in the sample code below, you will notice that above the code for [Sobel](#) function there is also code for the [Scharr](#) function commented. Uncommenting it (and obviously commenting the Sobel stuff) should give you an idea of how this function works.

3.1-3.2 High-Pass Filter: Scharr Filter

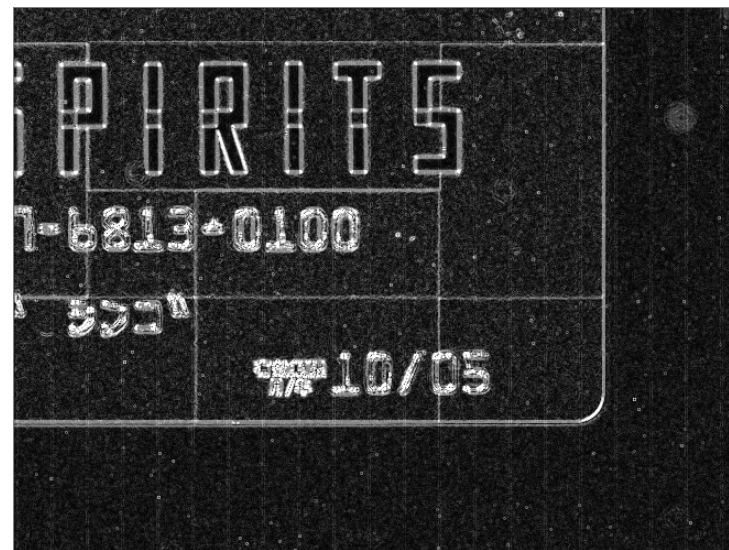
Input image



Sobel



Scharr



3.1-3.3 High-Pass Filter: Canny Edge Detector

4-connected component
8-connected component

➤ Canny Algorithm Steps:

1. Image Convert to Grayscale

2. **Smoothing:** Noise Reduction (Smoothing)

3. **Edge Detection:** Compute Gradient Magnitude and Angle using Sobel

4. **Thinning:** Non-Maximum Suppression

5. **Connectivity:** Hysteresis Thresholding: Low and High Thresholds

➤ OpenCV Canny Function Call

➤ Edge detection performance:

-Wavelets edge detection (slow) > canny > sobel (fast)

3.1-3.3 High-Pass Filter: Canny Edge Detector

```
➤ Void Canny(InputArray detected_edges, OutputArray detected_edges,  
             double lowThreshold, double highThreshold, int kernel_size )
```

Parameter:

- 1) **detected_edges**: Source image, grayscale
- 2) **detected_edges**: Output of the detector (can be the same as the input)
- 3) **lowThreshold**: The value entered by the user moving the Trackbar
- 4) **highThreshold**: Set in the program **as three times** the lower threshold
(following Canny's recommendation)
- 5) **kernel_size**: We defined it to be 3 (the size of the Sobel kernel to be used internally)

Ex:

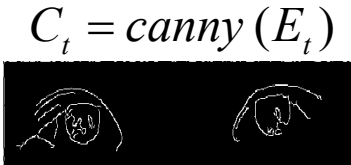
-1	0	+1
-2	0	+2
-1	0	+1

+1	+2	+1
0	0	0
-1	-2	-1

Kernel = window(表格) = filter



Input grayscale image



Output Edge image

$$C_t(x, y) = \begin{cases} 1, & \text{edge} \\ 0, & \text{no - edge} \end{cases}$$

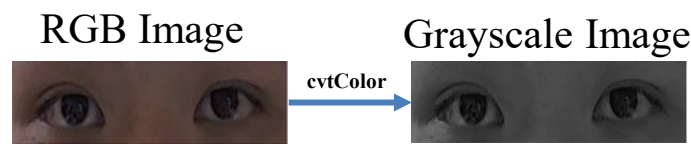
$$C_t(x, y) = \begin{cases} D, & \text{edge (1 ~ 255 or ex. 32 ~ 255)} \\ 0, & \text{no - edge} \end{cases}$$

accumulated
magnitude

theta

3.1-3.3 High-Pass Filter: Canny Edge Detector (1/3)

1. Image Convert to Grayscale



`cvtColor( image, gray_image, CV_BGR2GRAY )`
Parameters:
1) **image** – a source image (image)
2) **Gray_image** - a destination image (gray_image),
in which we will save the converted image.
3) **CV_BGR2GRAY** - an additional parameter that indicates
what kind of transformation will be performed.

High-pass filter, sum=0.0

3. Compute Gradient Magnitude and Angle

- Compute the derivatives ($D_x(x, y)$ and $D_y(x, y)$) of the image in the x and y directions.
Ex: Sobel filter

$D_y(x, y) =$

-1	0	+1
-2	0	+2
-1	0	+1

$D_x(x, y) =$

+1	+2	+1
0	0	0
-1	-2	-1

OpenCV 2.4.8.0 documentation » OpenCV Tutorials :

$$D_y = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad D_x = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} \quad G = \sqrt{G_x^2 + G_y^2}$$
$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

- Low-pass filter, sum=1.0
2. Noise Reduction (Smoothing)
- $$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

-Note that this 5×5 filter is roughly equivalent to the Gaussian filter

J: -Usually 3×3 is enough, normally, 7×7 is too big. We don't want it filters out the high-frequency components. So wavelet is one solution but not in this case.

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

$\frac{1}{159} \times$

2	4	5	4	2
4	9	12	9	4
5	12	15	12	5
4	9	12	9	4
2	4	5	4	2

J:?? Should D normalizes between 0~255 in order to later threshold?

- Compute the gradient magnitude:

$$D = \sqrt{D_x^2(x, y) + D_y^2(x, y)}$$

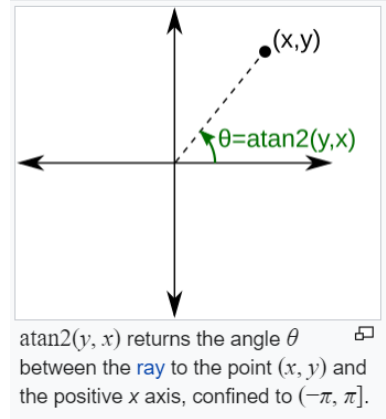
- Compute the angle of the gradient:

$$\theta = \arctan\left(\frac{D_y(x, y)}{D_x(x, y)}\right)$$

If $x > 0$, the angle is given by:

$$\theta = \text{atan2}(y, x) = \arctan\left(\frac{y}{x}\right)$$

$$x = r \cos \theta, \\ y = r \sin \theta.$$



$\text{atan2}(y, x)$ returns a single value θ such that $-\pi < \theta \leq \pi$ and, for $r = \sqrt{x^2 + y^2}$,

3.1-3.3 High-Pass Filter: Canny Edge Detector (2/3)

4. Non-Maximum Suppression (thinning)

➤ Goal: The edges it finds can be either very thick or very narrow depending on the intensity across the edge and how much the image was blurred first.

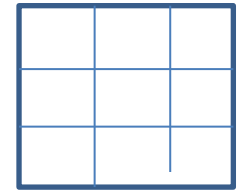
➤ Non-Maximum Suppression:

Three pixels in a 3×3 around pixel (x, y) are examined, there are 4 conditions:

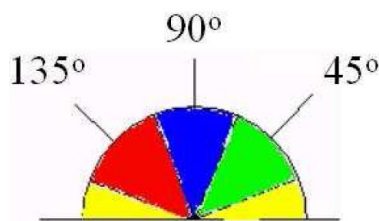
- If $\theta'(x, y) = 0^\circ$, then the pixels $(x + 1, y)$, (x, y) , and $(x - 1, y)$ are examined.
- If $\theta'(x, y) = 90^\circ$, then the pixels $(x, y + 1)$, (x, y) , and $(x, y - 1)$ are examined.
- If $\theta'(x, y) = 45^\circ$, then the pixels $(x + 1, y + 1)$, (x, y) , and $(x - 1, y - 1)$ are examined.
- If $\theta'(x, y) = 135^\circ$, then the pixels $(x + 1, y - 1)$, (x, y) , and $(x - 1, y + 1)$ are examined.

That is, whether the center pixel is the local max.

- If yes, keep it,
- else delete it.



8-connective components
4-connective components



Compute θ' by rounding the angle θ to one of four directions

0° , 45° , 90° , or 135° .

Obviously for edges, $180^\circ = 0^\circ$, $225^\circ = 45^\circ$, etc.

➤ Choose edge pixel :

- If pixel (x, y) has the highest gradient magnitude of the three pixels examined **for any one condition**, it is kept as an edge.
- If one of the other two pixels has a higher gradient magnitude, then pixel (x, y) is not on the “center” of the edge and should not be classified as an edge pixel.(如果周圍比自己大就忽略)

J: Will this kind of judgment cause slow performance?

3.1-3.3 High-Pass Filter: Canny Edge Detector (3/3)

5. Hysteresis Thresholding: Two thresholds – t_{low} and t_{high} (connectivity)

- Existing problem (solve by canny): A simple threshold may actually remove valid parts of a connected edge, leaving a disconnected final edge image.

J: It wants to keep connected edge/components.

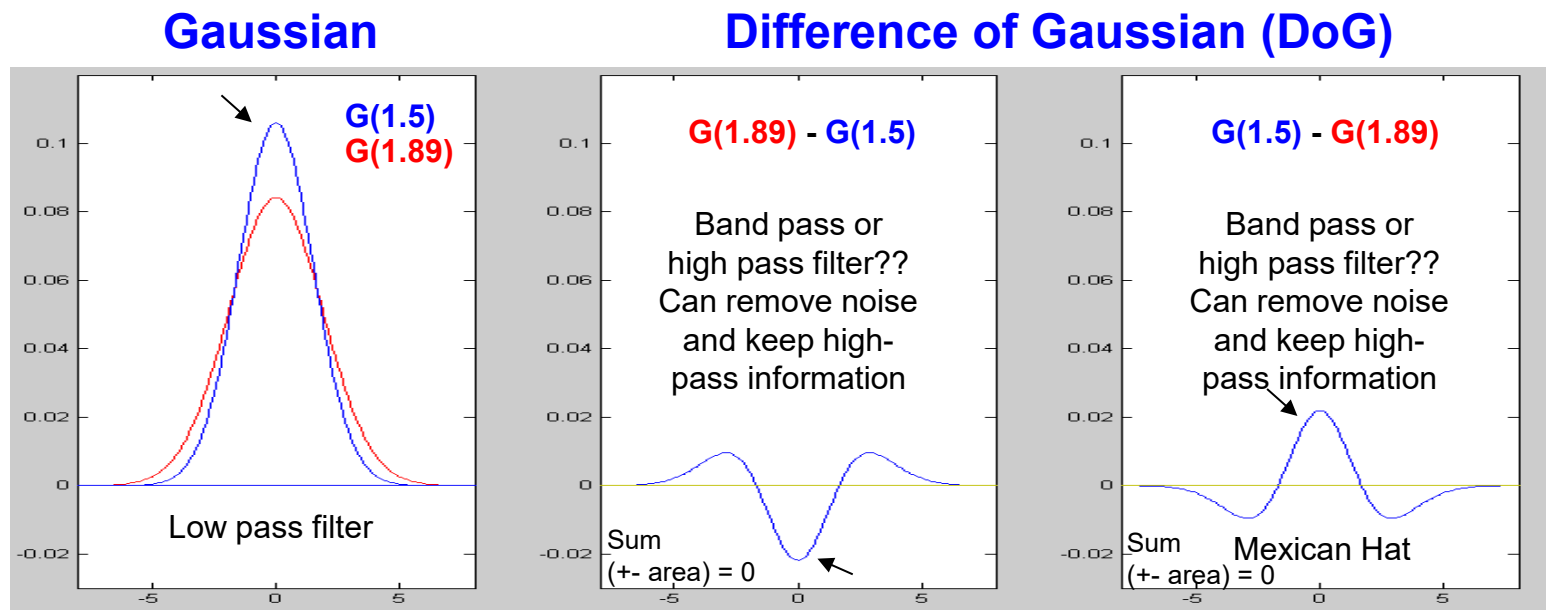
➤ Hysteresis Thresholding

- If pixel (x, y) has gradient magnitude less than t_{low} discard the edge (write out black).
- If pixel (x, y) has gradient magnitude greater than t_{high} keep the edge (write out white).
- If pixel (x, y) has gradient magnitude between t_{low} and t_{high} and any of its neighbors in a 3×3 region around it have gradient magnitudes greater than t_{high} , keep the edge (write out white).
- If none of pixel (x, y) 's neighbors have high gradient magnitudes but at least one falls between t_{low} and t_{high} , search the 5×5 region to see if any of these pixels have a magnitude greater than t_{high} . If so, keep the edge (write out white).
- Else, discard the edge (write out black).

3.1-3.4 High-Pass Filter: Difference of Gaussian

◆ Difference of Gaussian (DoG)

- For **scale invariance**, search for stable features across all possible scales using a continuous function of scale, such as Gaussian.
- SIFT uses **DoG filter** for scale space because it is efficient and as stable as scale-normalized Laplacian of Gaussian (LOG).



低通與高通濾波器

- 高頻(high-frequency components)：短距離內灰階值變化值大，intensity gradient 大。(例：texture、邊緣或雜訊)。
- 低頻(low-frequency components)：短距離內灰階值變化值小，intensity gradient 小。(例：textureless)
- 高通濾波器 (high-pass filter)：保持高頻部分，減少低頻部分。(係數總和為0)(例如 Sobel 或 Canny)
- 低通濾波器 (low-pass filter)：保持低頻部分，減少高頻部分。Smooth (係數總和為1)(例如 Gaussian Filter)

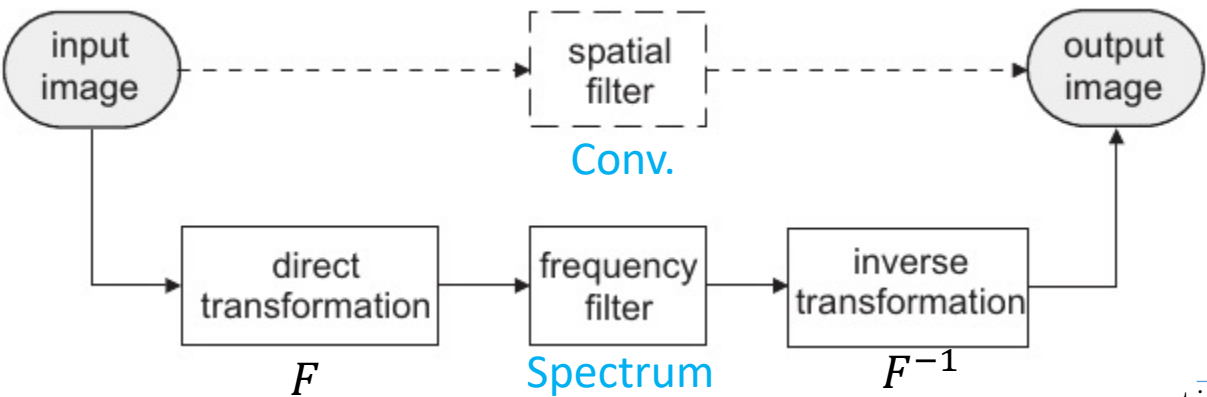
Example

high-pass filter: Sum = 0	low-pass filter: Sum = 1
$\begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}$	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

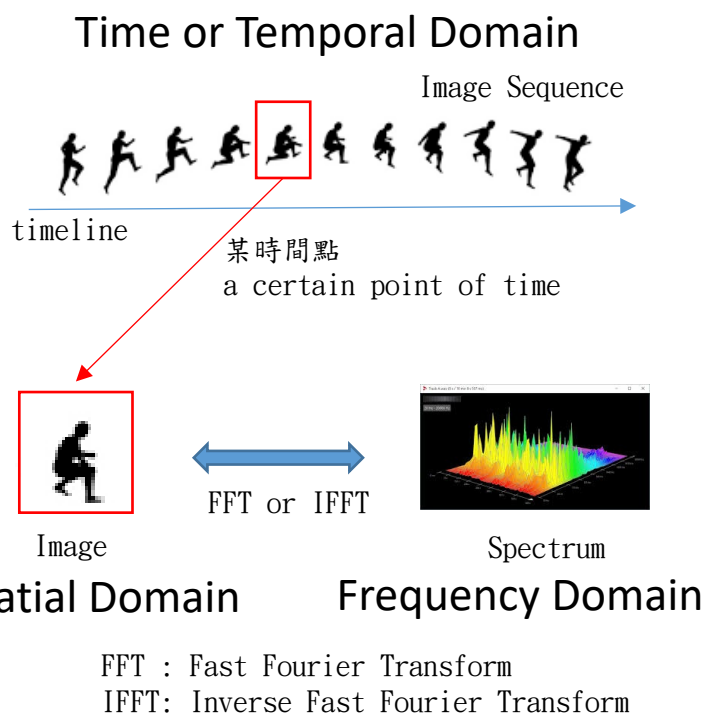
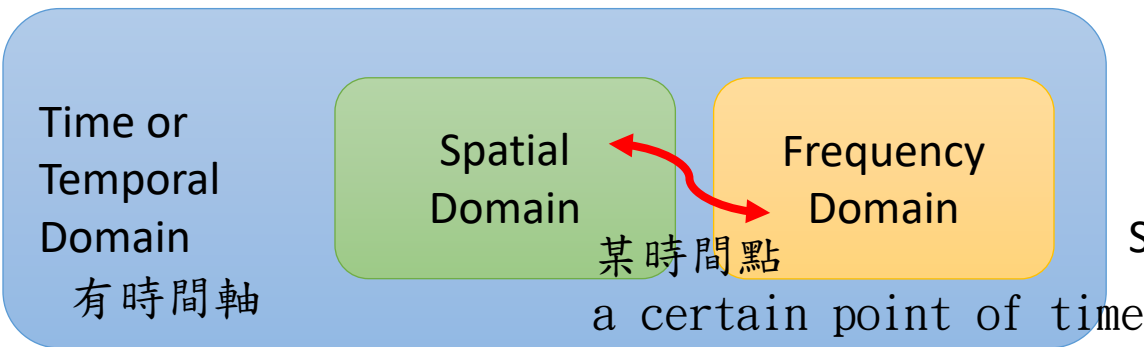
3.1-3.5 Spatial Domain Vs. Frequency Domain Vs. Temporal Domain (1/4)

0. Brief idea:

- Image processing is filtration of 2D signals.



- Domain (Space) Knowledge:

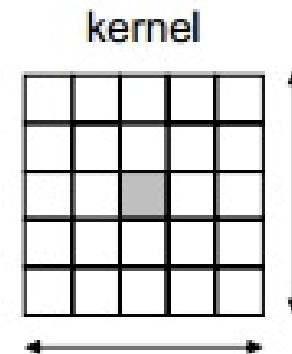


Sam

3.1-3.5 Spatial Domain Vs. Frequency Domain Vs. Temporal Domain (2/4)

1. Spatial Domain:

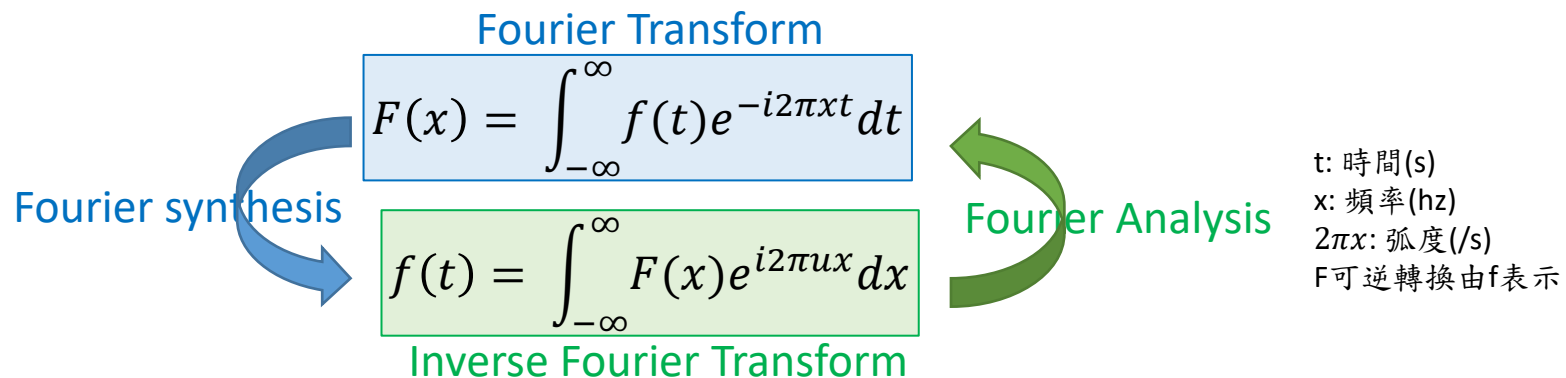
- Definition
 - is **manipulating or changing an image representing an object in space** to enhance the image for a given application.
 - Techniques are based on direct manipulation of pixels in an image.
 - Used for **filtering basics, smoothing filters, sharpening filters, unsharp masking.**



3.1-3.5 Spatial Domain Vs. Frequency Domain Vs. Temporal Domain (3/4)

2. Frequency Domain:

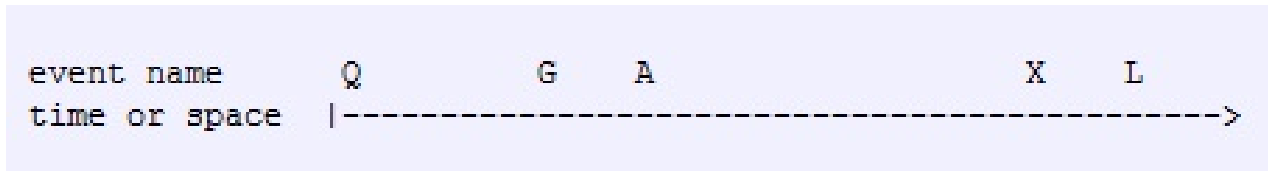
- Definition:
 - Techniques are based on **modifying the spectral transform** of an image
 - Transform the image to its frequency representation
 - Perform image processing
 - Compute **inverse transform back to the spatial domain**
- Technique
 - **Fourier Transform**: Function that are **not periodic** (but with **finite area** under the curve) can be expressed as **the integral of sines and cosines multiplied** by a weight function (複指數函數的積分).



3.1-3.5 Spatial Domain Vs. Frequency Domain Vs. Temporal Domain (4/4)

3. Temporal Domain:

- Definition:
 - Temporality is described here as the **ratios** of, or **relative** intervals between events. The temporal domain carries no information about frequency or sequence. One way to represent the temporal domain from the time-line is as below:



- 相對於每個點之間的時間或空間差

Q:	G=2	A=3	X=7	L=8
G:	Q=2	A=1	X=5	L=6
A:	G=1	Q=3	X=4	L=5
X:	Q=7	G=5	A=4	L=1
L:	Q=8	G=6	A=5	X=1

- The only information carried in the temporal domain are the distances between events relative to the distances between other events; for example "There is twice as much time between A and X as there is between G and Q". The actual intervals could be microseconds, years, or centuries among other things. **The temporal representation retains no hint of this.**

Content: Topic1. Similarity Measure (相似度量測)

1.0 Computer Vision - Camera Model: Between 3D and 2D

1.1 2D Image

1.2 Camera Model Btw 3D Object and 2D Image

1.3 From Linear Combination to Homography $Ax=b$ (Vector, Matrix)

2.0 Computer Vision (CV) and AI: Similarity Measure = Loss Function

2.1 Sum-of-Squared Differences (SSD) and Mean Squared Error (MSE)

2.2 Normalized Correlation Coefficient (NCC) and Cross Entropy (CE)

2.3 AI Bayesian Decision Rule - Maximum a Posteriori (MAP) Probability with Gaussian Model

3.0 Image Processing for Deep Learning: Filter, Correlation and Convolution

3.1 Low-Pass Filter - Denoise and High-Pass Filter - Edge Detection

3.2 Correlation Vs. Convolution

3.3 Multi-Resolution - Downsampling (Encoder) and Upsampling (Decoder)

3.2 Correlation Vs. Convolution

↔ Correlation Matrix Vs. Covariance Matrix



3.2-1 Correlation Vs. Convolution: Template Matching + Scanline

1. Template matching is a technique for finding areas of an image that is **similar** or **match** to a template by **similarity measure**
2. The operation procedure:
 - **Image and template**
 - We need two primary components: **Source image** (image) and **template image** (template)
 - **Scanline (sliding)**
 - To identify the matching area, we have to compare the template image against the source image by **sliding** it.
 - By sliding (scanline), we mean moving the patch one pixel at a time (**left to right, up to down**). **move = shift = stride**



Source Image



Template



Compare the template image against the source image by sliding it.

3.2-1 Correlation Vs. Convolution: Equation (1/2)

1. Correlation Vs. Convolution

1) Correlation: without rotation (flipping) the filter h

• 數學式:

$$\overset{\text{Pixel}(x,y)}{G[x,y]} = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} h[u,v] \cdot F(x+u, y+v)$$

F: Image

h: filter, size $N \times N$

- Summation Loop方向: 從左到右, 從上到下 (From left to right, from top to bottom.)

2) Convolution: rotate/flip the filter h

• 數學式:

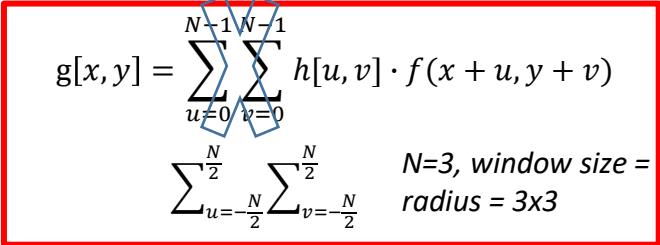
$$\begin{aligned} G[x,y] &= \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} h[u,v] \cdot F(x-u, y-v) \\ &= \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} h'[u,v] \cdot F(x+u, y+v) \end{aligned}$$

F: Image

h: filter, size $N \times N$

h' : filter which is rotated/flipped

- Summation Loop方向: 從右到左, 從下到上 (From right to left, from bottom to top.)


$$g[x,y] = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} h[u,v] \cdot f(x+u, y+v)$$
$$\sum_{u=-\frac{N}{2}}^{\frac{N}{2}} \sum_{v=-\frac{N}{2}}^{\frac{N}{2}} \quad N=3, \text{ window size} = \text{radius} = 3 \times 3$$

3) 比較: filter相反, 但當filter的x軸與y軸對稱時, 在2-D的運算上就沒有差別了

Compare: The opposite of the filter, but when the x-axis of the filter is symmetric with the y-axis, there is no difference in the 2-D operation.

3.2-1 Correlation Vs. Convolution: Equation (2/2)

2. Correlation Matrix Vs. Convolution Matrix

1) Correlation Matrix

0	0	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0
0	0	0	0	0

Image

·

a	b	c
d	e	f
g	h	i

filter

=

0	0	0	0	0
0	i	h	g	0
0	f	e	d	0
0	c	b	a	0
0	0	0	0	0

Image'

2) Convolution Matrix

0	0	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0
0	0	0	0	0

Image

·

i	h	g
f	e	d
c	b	a

filter'

=

0	0	0	0	0
0	a	b	c	0
0	d	e	f	0
0	g	h	i	0
0	0	0	0	0

Image''

Convolution 中 filter 先 rotate
因此成像就會變正的
convolution middle filter first rotate
So imaging will be corrected.

3.2-2 Correlation and Convolution (1/2)

Padding =
Radius of Kernel

1. 1-D filter or window

Description

- 1) 1-D function f & filter w
 $len(f) = 8$ & $len(w) = 5$
- 2) Starting point
- 3) Zero padding and correlation at 1st point
 $length = len(f) + 2(len(w) - 1)$
- 4) 2nd correlation value
- 5) Shift by 4
- 6) Final point
- 7) Full correlation result
 $length = len(f) + len(w) - 1$
- 8) Cropping with $len(f)$

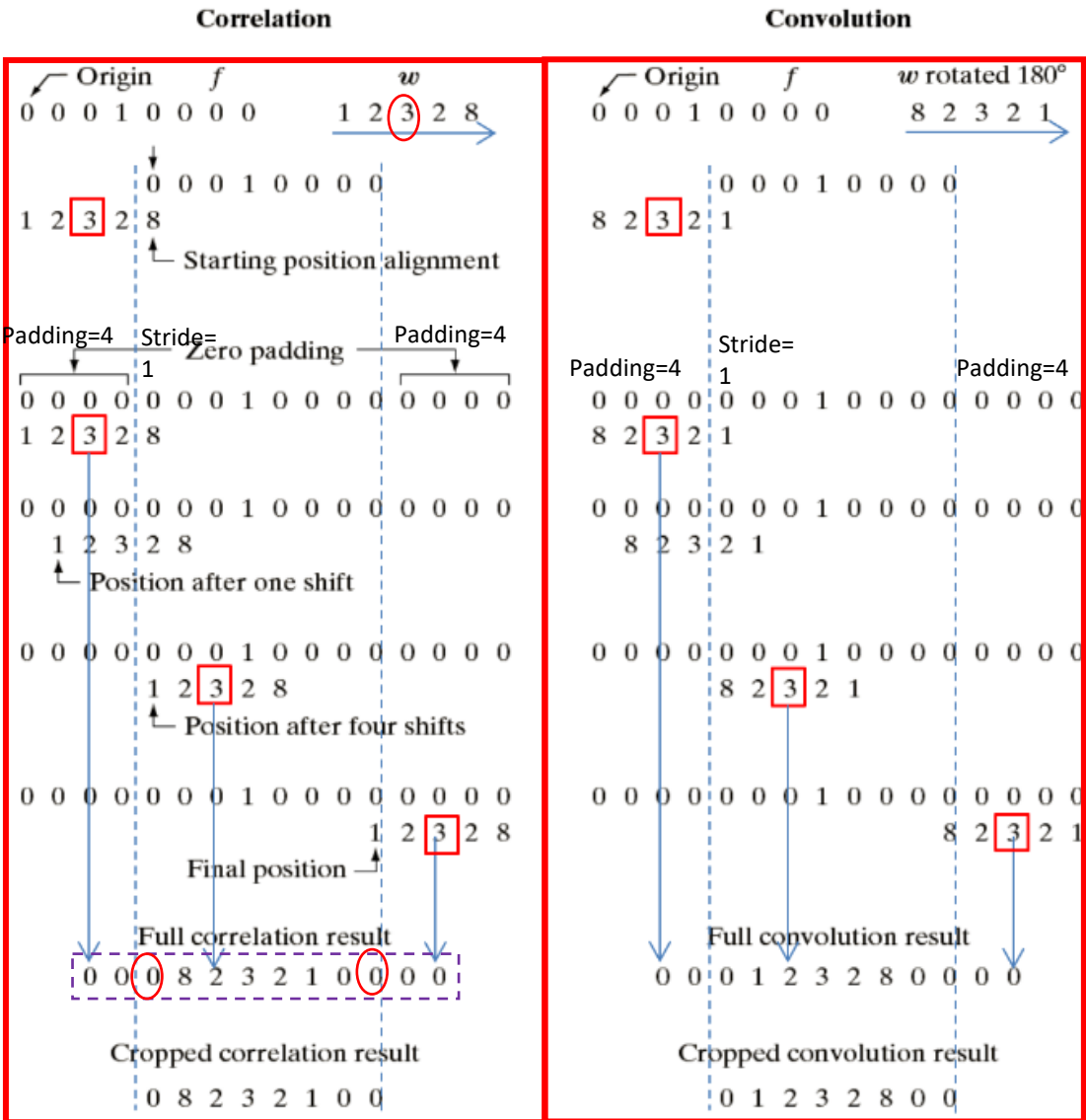
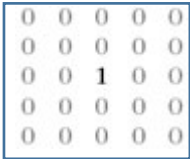


FIGURE 3.29 Illustration of 1-D correlation and convolution of a filter with a discrete unit impulse. Note that correlation and convolution are functions of displacement

3.2-2 Correlation and Convolution (2/2)

2. 2-D filter or window

1) 2-D function $f(x, y)$ & filter $w(x, y)$



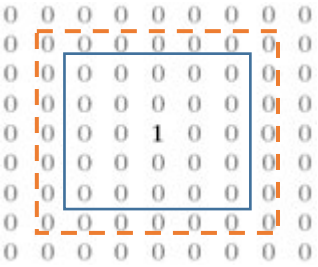
$f(x, y)$



$w(x, y)$

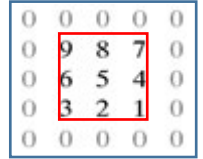
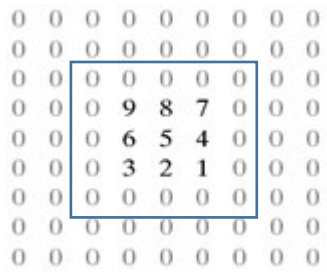
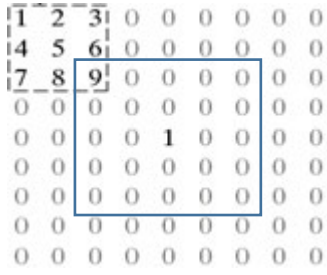
2) Zero padding

Stride=1
✗ Padding=2 → =1 is enough
Padding =
Radius of Kernel



Length=
 $\text{len}(f)+2 \times (\text{len}(w)-1)$
 $5+2 \times (3-1)=9$

3) Correlation: Initial position for $w \rightarrow$ Full result $\rightarrow$ Cropped result



4) Convolution: Rotated $w \rightarrow$ Full result $\rightarrow$ Cropped result

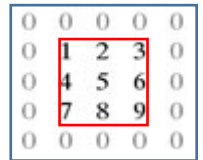
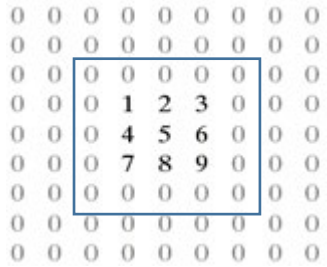
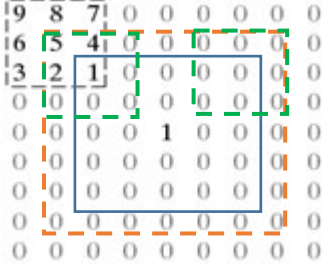


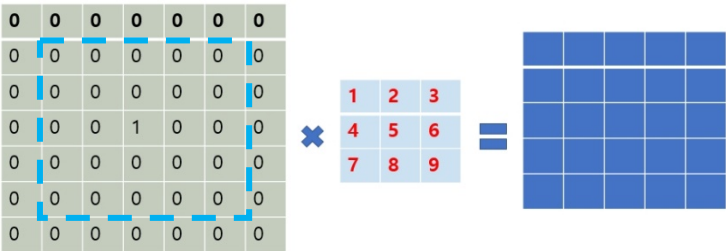
FIGURE 3.30 Correlation (middle row) and convolution (last row) of a 2-D filter with a 2-D discrete, unit impulse

3.2-3 Correlation Matrix Vs. Convolution Matrix (1/2-1/2)

2. Correlation Matrix Vs. Convolution Matrix

1) Correlation Matrix

1. Initial Status

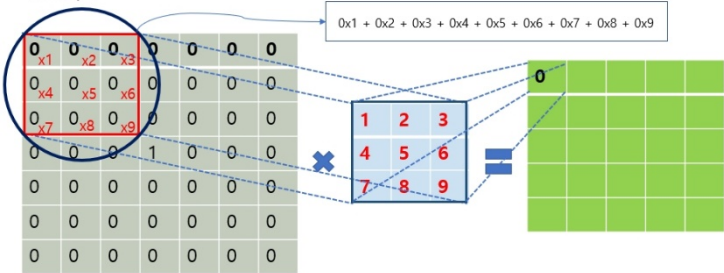


Input

template

Output

2. Step1

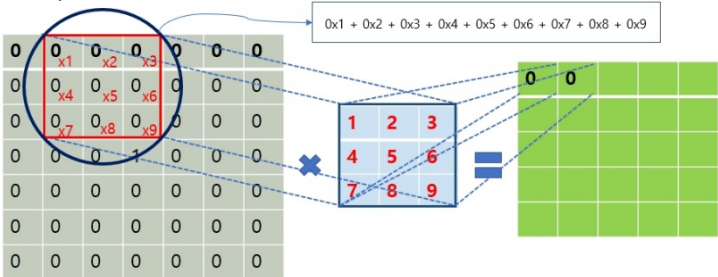


Input

template

Output

2. Step2

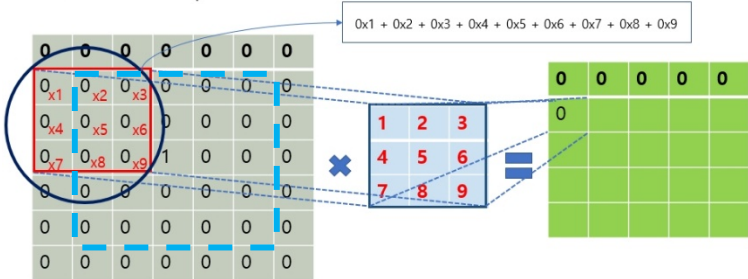


Input

template

Output

3. continue To Step6

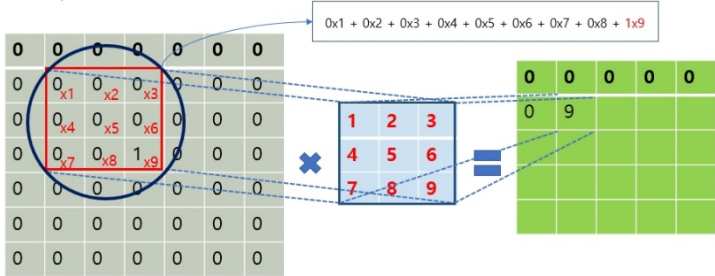


Input

template

Output

4. Step7

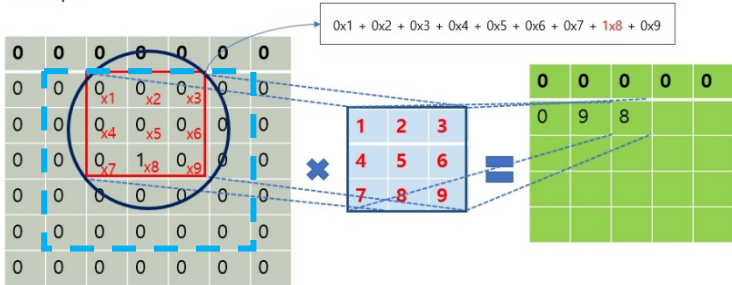


Input

template

Output

5. Step8



Input

template

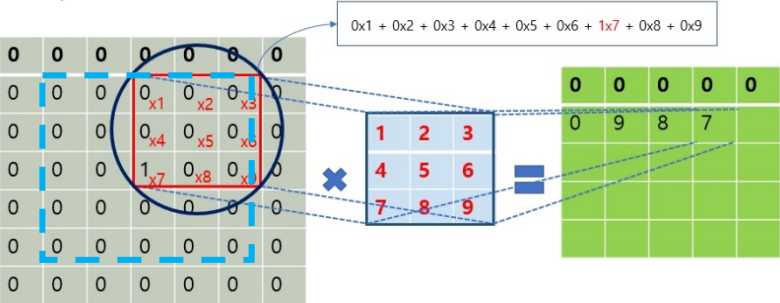
Output

3.2-3 Correlation Matrix Vs. Convolution Matrix (1/2-2/2)

2. Correlation Matrix Vs. Convolution Matrix

1) Correlation Matrix

6. Step9

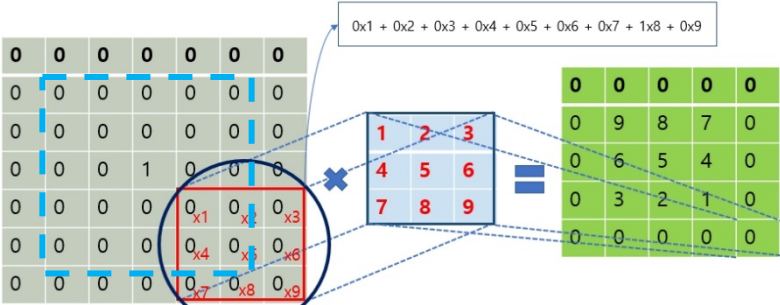


Input

template

Output

7. Result



Input

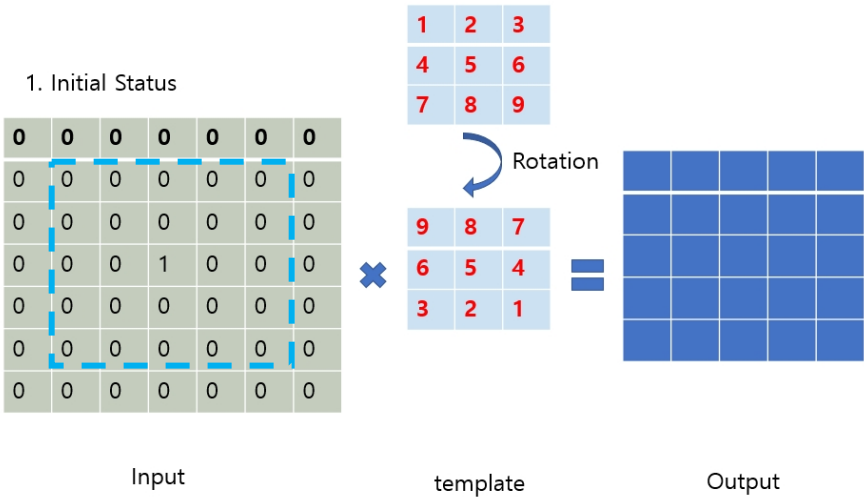
template

Output

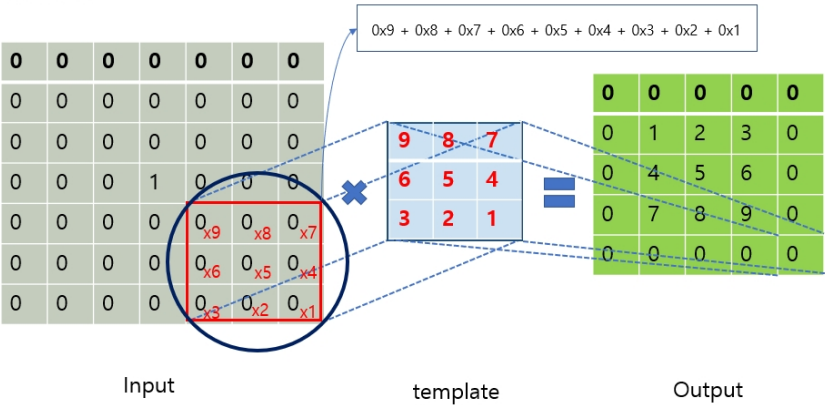
3.2-3 Correlation Matrix Vs. Convolution Matrix (2/2)

2. Correlation Matrix Vs. Convolution Matrix

2) Convolution Matrix



2. Result



Content: Topic1. Similarity Measure (相似度量測)

1.0 Computer Vision - Camera Model: Between 3D and 2D

1.1 2D Image

1.2 Camera Model Btw 3D Object and 2D Image

1.3 From Linear Combination to Homography $Ax=b$ (Vector, Matrix)

2.0 Computer Vision (CV) and AI: Similarity Measure = Loss Function

2.1 Sum-of-Squared Differences (SSD) and Mean Squared Error (MSE)

2.2 Normalized Correlation Coefficient (NCC) and Cross Entropy (CE)

2.3 AI Bayesian Decision Rule - Maximum a Posteriori (MAP) Probability with Gaussian Model

3.0 Image Processing for Deep Learning: Filter, Correlation and Convolution

3.1 Low-Pass Filter - Denoise and High-Pass Filter - Edge Detection

3.2 Correlation Vs. Convolution

3.3 Multi-Resolution - Downsampling (Encoder) and Upsampling (Decoder)

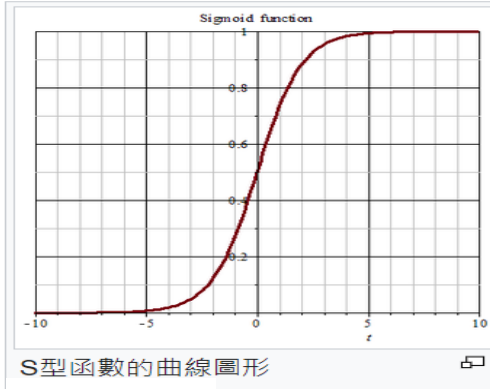
3.3 Multi-Resolution – Downsampling (Encoder) and Upsampling (Decoder)

3.3-0 Deep Learning - CNN: AI MLP-BP

Convolutional
Neural Network

➤ I/P MLP
- Traditional AI:
I/P: Raw (image) data

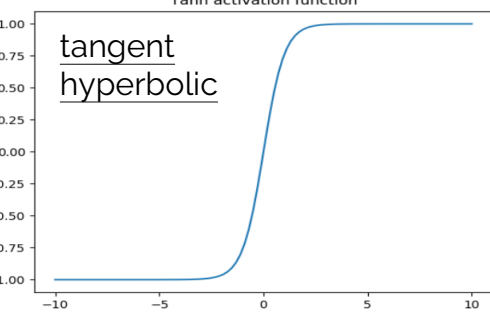
- Deep Learning:
I/P: Extracted feature maps



Normalization for 4 class probability:

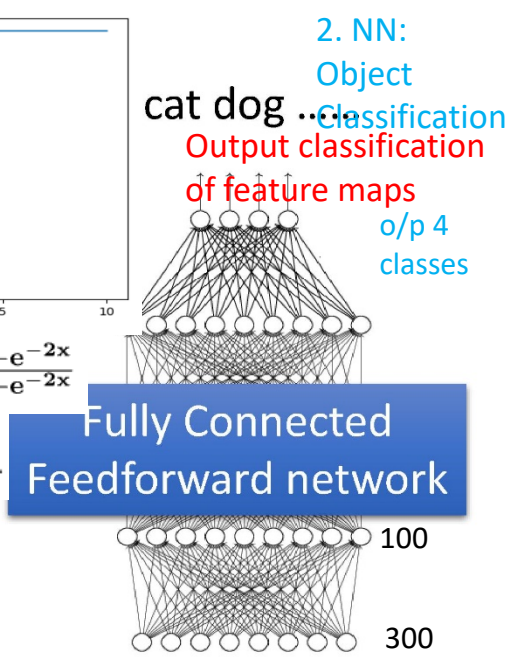
- 1) Sigmoid[0.0,1.0] or tanh[-1.0, 1.0] for 1 class
- 2) Softmax for more than 1 class

$$S(t) = \frac{1}{1 + e^{-t}}$$



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$
$$= \frac{2 - (1 + e^{-2x})}{1 + e^{-2x}} = \frac{2}{1 + e^{-2x}} - 1 = 2s(2x) - 1$$

The whole CNN

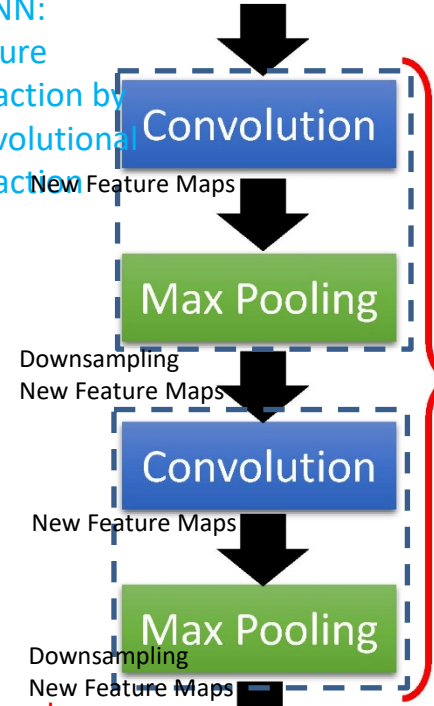


2. NN: Object Classification

1. CNN: Feature Extraction by Convolutional Extraction



Input 600x400x3c image



Conv. + BN + Activation

600x400x8c
Downsampling:
1) Max pooling 2) Stride

Can repeat
300x200x8c
many times

300x200x16c

150x100x16c

拉平 matrix to vector

Flatten

1x(150x100x16c)



3.3-0 Deep Learning – CNN: VGG16 or VGG19 2015 – Very Deep CNN

(13+3 or 16+3 Layers (as 5 + 3 Layers))

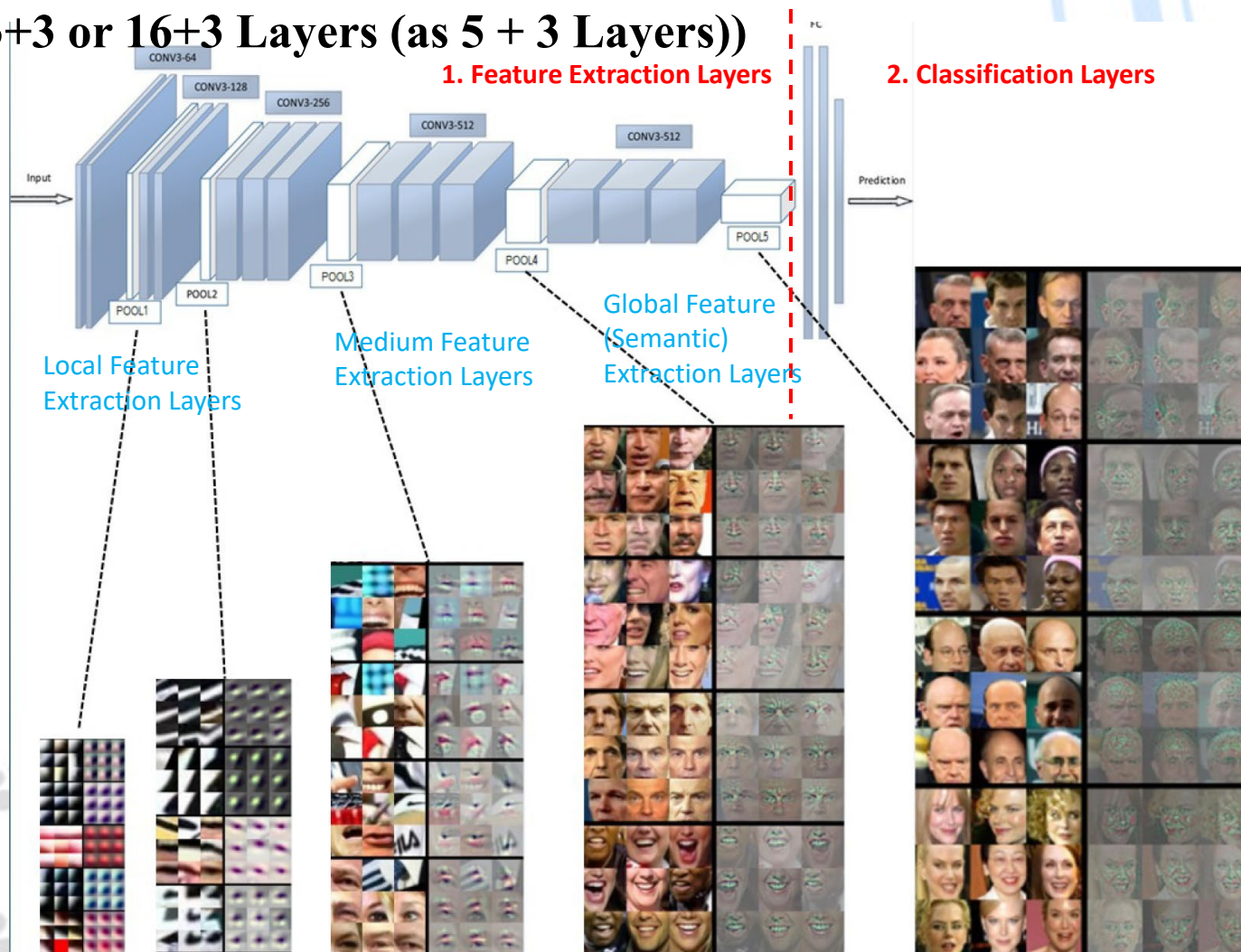


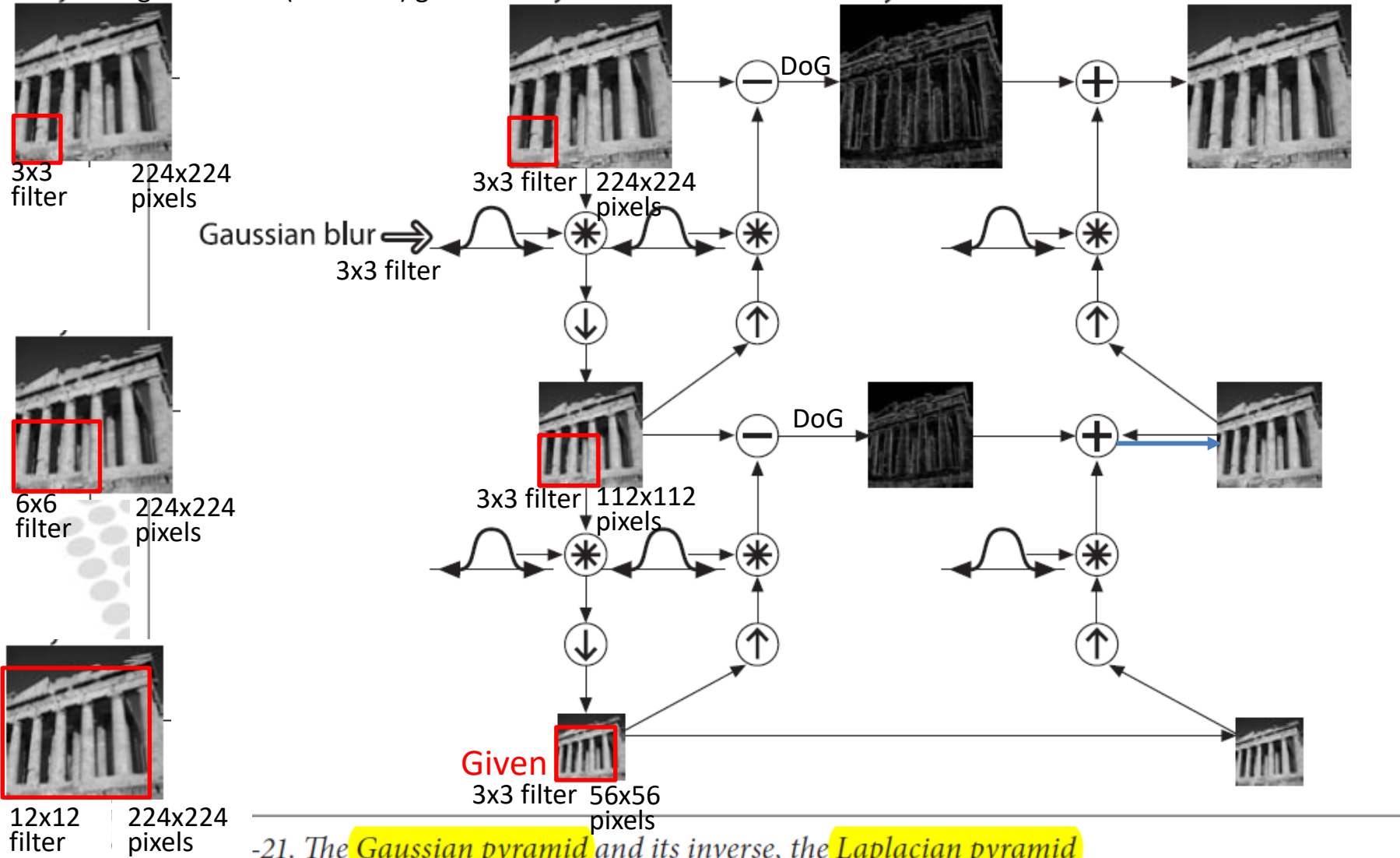
Fig. 2. The hierarchical architecture that stitches together pixels into invariant face representation. Deep model consists of multiple layers of simulated neurons that convolute and pool input, during which the receptive-field size of simulated neurons are continually enlarged to integrate the low-level primary elements into multifarious facial attributes, finally feeding the data forward to one or more fully connected layer at the top of the network. The output is a compressed feature vector that represent the face. Such deep representation is widely considered the state-of-the-art technique for face recognition.

3.3-1 Multi-Resolution – (1/2)

Downsampling (Encoder) and Upsampling (Decoder)

Encoder by downsampling:

- 1) Reduce the parameters
- 2) Increase the reception field from local to geometrical (semantic) global

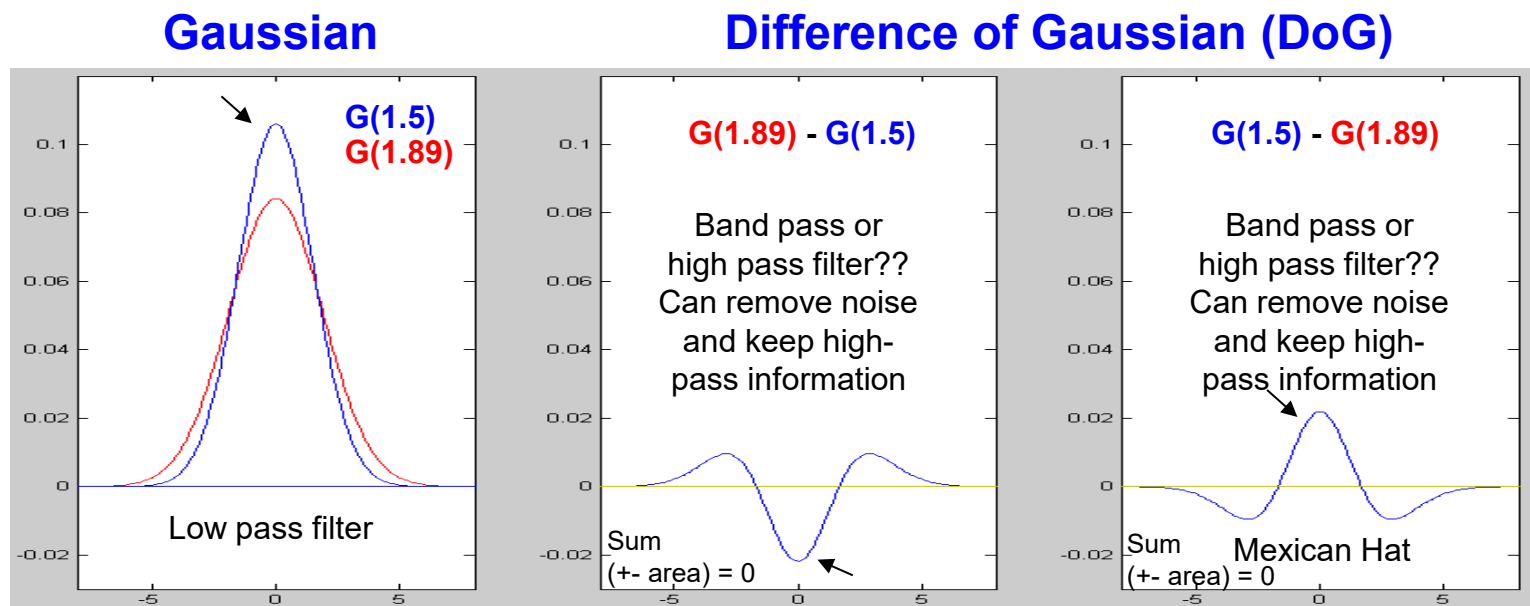


3.3-1 Multi-Resolution – (2/2)

Downsampling (Encoder) and Upsampling (Decoder)

◆ Difference of Gaussian (DoG)

- For **scale invariance**, search for stable features across all possible scales using a continuous function of scale, such as Gaussian.
- SIFT uses **DoG filter** for scale space because it is efficient and as stable as scale-normalized Laplacian of Gaussian (LOG).



低通與高通濾波器

- 高頻(high-frequency components)：短距離內灰階值變化值大，intensity gradient 大。(例：texture、邊緣或雜訊)。
- 低頻(low-frequency components)：短距離內灰階值變化值小，intensity gradient 小。(例：textureless)
- 高通濾波器 (high-pass filter)：保持高頻部分，減少低頻部分。(係數總和為0)(例如 Sobel 或 Canny)
- 低通濾波器 (low-pass filter)：保持低頻部分，減少高頻部分。Smooth (係數總和為1)(例如 Gaussian Filter)

Example

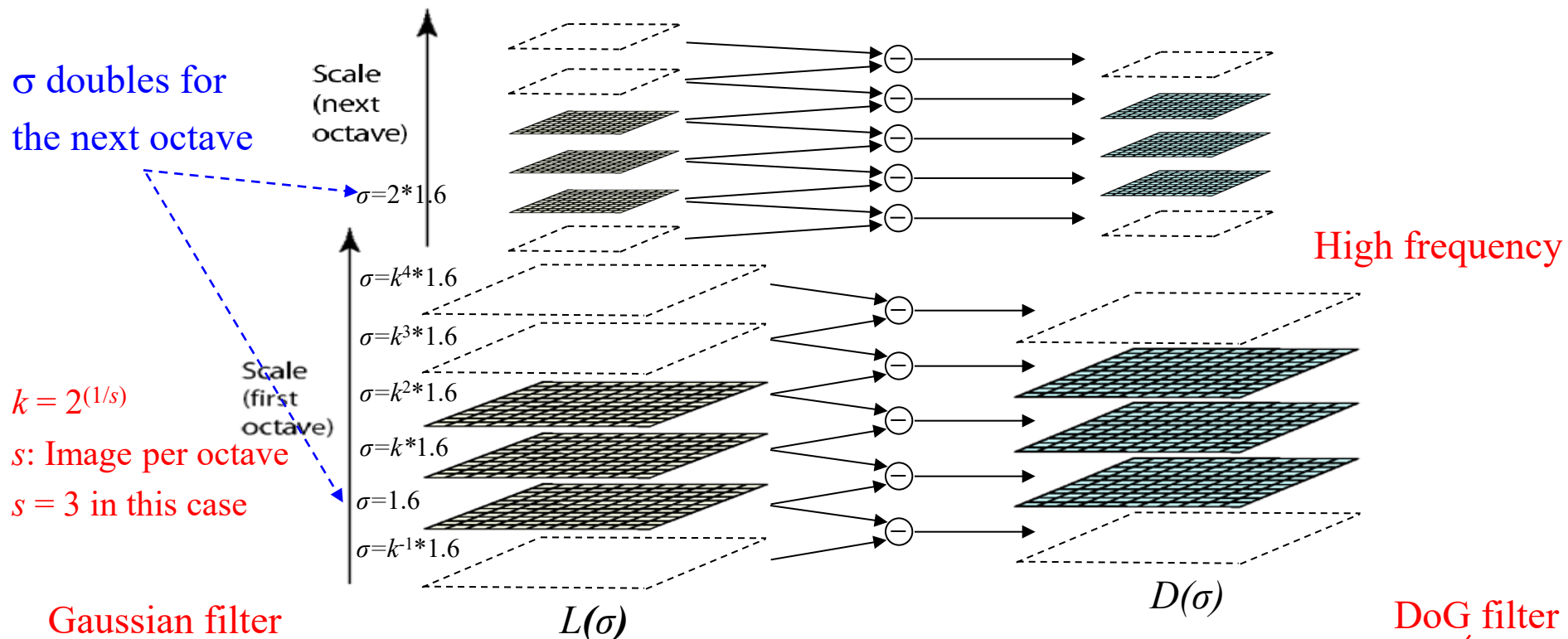
high-pass filter: Sum = 0 low-pass filter: Sum = 1

$$\begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix} \quad \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

3.3-1 Difference of Gaussian (DoG) filter (1/2)

Dividing into octave is for efficiency only

⋮ Until image size < 32x32



$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

$$G(x, y, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(x^2 + y^2)}{2\sigma^2} \right)$$

$$\begin{aligned} D(x, y, k^{(n-1)}\sigma) & \quad n = 1, 2, \dots, s \\ &= (G(x, y, k^n\sigma) - G(x, y, k^{(n-1)}\sigma)) * I(x, y) \\ &= L(x, y, k^n\sigma) - L(x, y, k^{(n-1)}\sigma) \end{aligned}$$

3.3-1 Difference of Gaussian (DoG) filter (2/2)



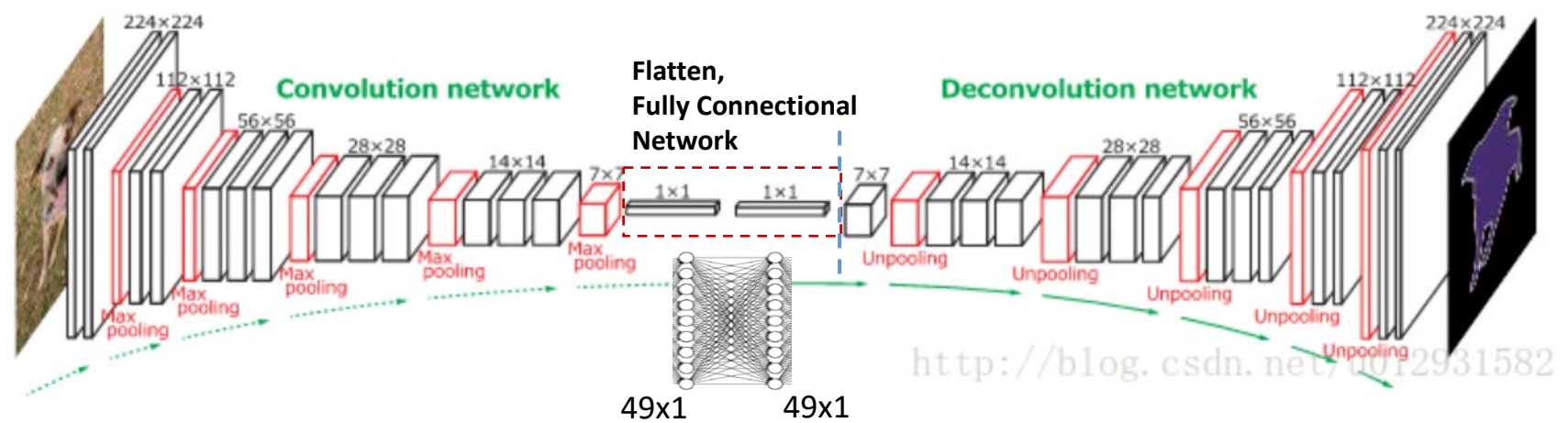
$L(\sigma)$



$D(\sigma)$

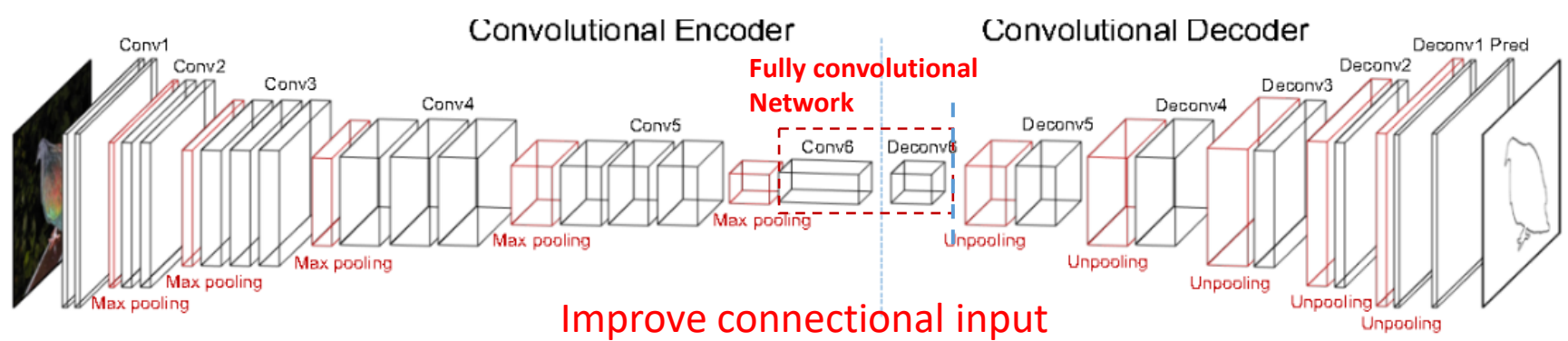
3.3-2 AutoEncoder – Convolutional Encoder and Decoder (1/3)

◆ AutoEncoder: Convolution Encoder Decoder Network



Reference paper: Noh, H., Hong, S., Han, B.\ 2015.\ Learning Deconvolution Network for Semantic Segmentation.\ arXiv e-prints arXiv:1505.04366.

Share



Improve connectional input problem: 1) Convolution, 2) SPP

Figure 2. Architecture of the proposed fully convolutional encoder-decoder network.

Reference paper: Yang, J., Price, B., Cohen, S., Lee, H., Yang, M.-H.\ 2016.\ Object Contour Detection with a Fully Convolutional Encoder-Decoder Network.\ arXiv e-prints arXiv:1603.04530.

3.3-2 U-Net Architecture (2/3)

Total the network has 23 convolutional layers.

U-Net Architecture: is Fully Convolutional Network (FCN) and consists of a 1) contracting path and 2. expansive path.

- (1) -3x3 Convolution
 -(Unpadded convolutions)
 - Activation function: ReLU
 -Stride: 1

1) Contracting path
 (Encoder)

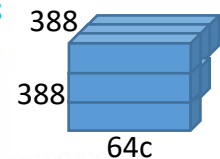
2) Expansive path
 (Decoder)

Local feature

(4) Concatenate— Crop (dash-region) and then Copy/concatenate features map form contracting path: 從 Contracting path 中 1 stage 最後一個convolution中feature map複製到 Expansive path 對應的feature map相同大小。

64+64(form first block)=128

(5) 1x1x64 fully convolution (FCN) and 388x388 filters: from 64 component feature vector to the desired number of classes $K=2$.



output segmentation map
 Test O/P:
 (388x388)x
 2 heat maps

Due to the unpadded convolutions, the output image is smaller than the input.
 (572*572->388*388)

(2) Max pooling

- Window size = 2x2
 -Stride = 2

Filter:(3x3x64c)x 128個

128 of 200x200 feature maps

(3) Up-Convolution

- (Unpadded convolutions)
 - Window size = 2x2
 - Stride = 2

~~BN~~

conv 3x3, ReLU
 (3x3xCh)x NumFilters
 copy and crop

max pool 2x2

up-conv 2x2
 (2x2xCh)x (Ch/2) NumFilters
 conv 1x1
 (1x1xCh)x NumFilters

Global feature

Drop-out

Filter:(3x3x1024c)x 512個

512 of 56x56:
 cropped
 feature maps
 from 64 to 56

concatenate

??reduce to Half or
 channel number

Filter:(2x2x1024)x 512個,

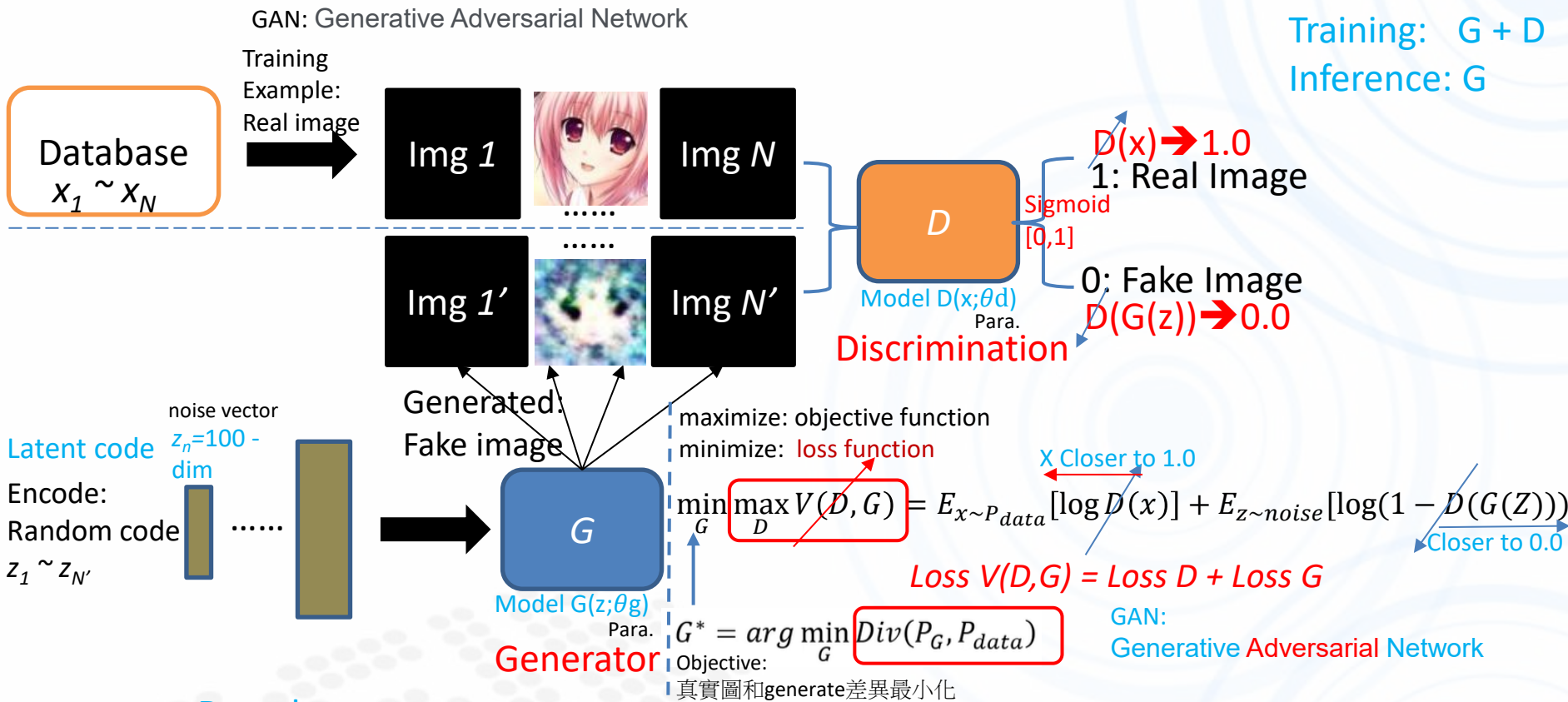
s=2

Code

??

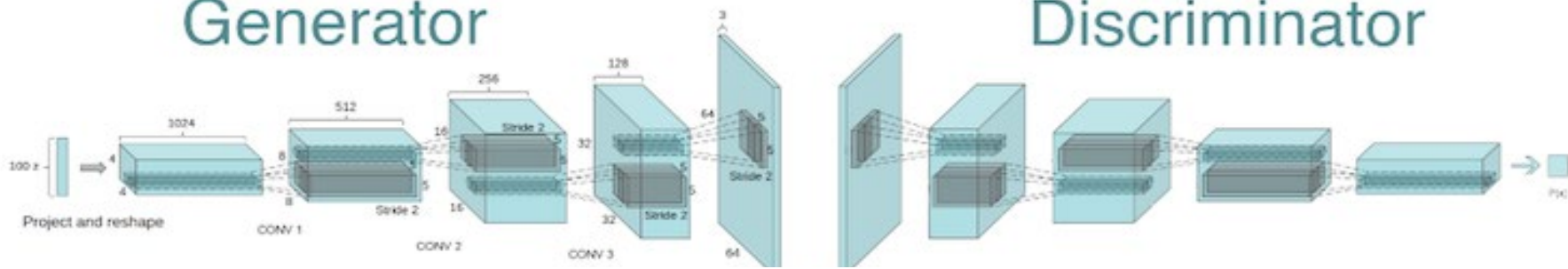
82

3.3-2 GPT - GAN for Loss Constraint (Regulation) Term and Prediction (3/3)



Decoder Generator

Encoder Discriminator



- GAN for loss constraint (regulation) term and prediction
 - ➔ Regulation term (Discrimination) to improve the recall/precision and stability
 - ➔ But training process is not easy convergence, so need pre-training

3.3-3 Resize or Interpolation for Upsampling or Downsampling (1/3)

destination image then the source will be resized to

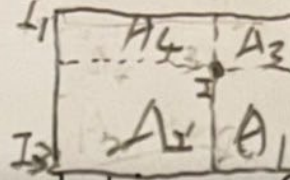
```
void cvResize(
    const CvArr*
    CvArr*
    int
);
```

src,

dst,

interpolation = CV_INTER_LINEAR

② Bilinear Interpolation



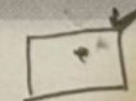
$$I = \sum_{i=1}^4 w_i$$

$$w_i = \frac{1}{4}$$

The last argument is the interpolation method, which
The other available options are shown in Table 5-4.

✓ Table 5-4. *cvResize()* interpolation options

Interpolation	Meaning
CV_INTER_NN	Nearest neighbor
CV_INTER_LINEAR	Bilinear
CV_INTER_AREA	Pixel area re-sampling
CV_INTER_CUBIC	Bicubic interpolation



$$d_1 = x_1$$

$$d_2 I_1$$

$$d_2 I_1$$

$$I' =$$

3.3-3 Resize or Interpolation for Upsampling or Downsampling (2/3)

LINEAR

① Linear interpolation 2x1

1, which defaults to linear interpolation.

5-4. I_1, I'_x, I_2

① x_1, x', x_2

th $\frac{I_1 - I'_x}{x_1 - x'} = \frac{I'_x - I_2}{x' - x_2}$

$d_1 = x_1 - x' = d_2 = x' - x_2$

$d_2 I_1 - d_1 I'_x = d_1 I'_x - d_2 I_2$

$d_2 I_1 + d_1 I_2 = (d_1 + d_2) I'_x$

$I'_x = \frac{d_2}{d_1 + d_2} I_1 + \frac{d_1}{d_1 + d_2} I_2$

weight 1

weight 2

Normalization

weight 2 $\in$ Prob

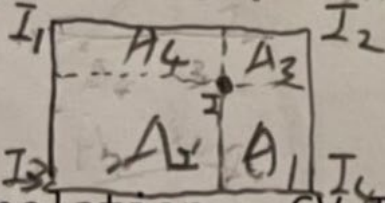
Priori

wx $\frac{1}{d}$ distance

3.3-3 Resize or Interpolation for Upsampling or Downsampling (3/3)

the source will be resized to fit into the ROI.

② Bilinear Interpolation 2x2



$$I = \sum_{i=1}^4 w_i I_i$$

$$w_i = \frac{A_i}{\sum_{j=1}^4 A_j}$$

Interpolation = CV_INTER_LINEAR

③ Linear interpolation 2x1

mesh

